



INDIAN INSTITUTE OF SCIENCE EDUCATION AND RESEARCH
BHOPAL

SUMMER RESEARCH INTERNSHIP REPORT

Benchmarking Computational Biology Tools Using Real and Simulated Data

Submitted by

Harshit Kumar Singh

Roll No: 7307

DEPARTMENT OF *BIOLOGICAL SCIENCES*

Indian Institute of Science Education and Research Bhopal

Under the supervision of

Dr. Nagarjun Vijay

Assistant Professor

Computational Evolutionary Genomics Lab

Department of Biological Sciences

IISER Bhopal

Duration: **1 June – 20 July 2026**

Submitted in *July 2026*

Certification

This is to certify that the work presented in this Summer Research Internship Report entitled “**Benchmarking Computational Biology Tools using Real and Simulated Data**” has been carried out by **Harshit Kumar Singh** (Roll No. 7307) under my supervision in the **Computational Evolutionary Genomics Laboratory**, Department of Biological Sciences, Indian Institute of Science Education and Research (IISER) Bhopal, during the period **1 June 2026 – 20 July 2026**.

To the best of my knowledge, the work presented in this report is original and was completed as part of the Summer Research Internship conducted under my supervision.

Date

Signature of PI
Dr. Nagarjun Vijay

Acknowledgement

I would like to express my sincere gratitude to Dr. Nagarjun Vijay, Assistant Professor, Department of Biological Sciences, Indian Institute of Science Education and Research (IISER) Bhopal, for providing me with the opportunity to undertake this Summer Research Internship in the Computational Evolutionary Genomics Laboratory. His guidance, constructive feedback, and insightful discussions throughout the internship were instrumental in shaping this work.

I particularly appreciate his mentoring approach, which encouraged independent thinking and discussions rather than simply providing solutions. This environment motivated me to explore problems critically, develop my own understanding of the computational methods, and approach research with greater confidence.

I am equally grateful to the Ph.D. scholars and members of the Computational Evolutionary Genomics Laboratory for their continuous support, and willingness to help whenever technical or conceptual challenges arose. Their collaborative environment made the learning process both engaging and enjoyable.

My sincere thanks are due to the Cluster Innovation Centre, University of Delhi, for encouraging undergraduate students to pursue research opportunities and for providing the academic foundation that made this internship possible.

Finally, I gratefully acknowledge everyone who, directly or indirectly, contributed to the successful completion of this internship and the development of LCRBenchmark.

Harshit Kumar Singh

Abstract

Low Complexity Regions (LCRs) are protein sequence segments enriched in a limited set of amino acids and are associated with diverse biological functions. Numerous computational methods have been developed for their detection; however, differences in their underlying algorithms often produce inconsistent predictions, making systematic benchmarking essential for comparative evaluation.

This internship, conducted under the project "Benchmarking Computational Biology Tools Using Real and Simulated Data," focused on reproducing and automating an existing benchmarking framework for LCR detection tools. Eight LCR detection tools were executed using thirteen configurations across the reference proteomes of seven model organisms. To enable uniform downstream analyses, heterogeneous tool outputs were standardized into a common 1-based BED format through custom Python scripts.

The benchmarking workflow was reconstructed in Python by developing scripts for data preprocessing, statistical analysis, figure generation, and automated report creation. The complete pipeline was subsequently automated using Snakemake, providing reproducible, dependency-aware execution while minimizing manual intervention. To improve accessibility, a Flask-based web application, LCRBenchmark, was developed to provide a graphical interface for workflow execution and result visualization. The application was containerized using Docker and deployed on the Render cloud platform.

The developed framework transforms a manually executed benchmarking methodology into a reproducible, modular, and user-friendly software platform. By integrating standardized data processing, workflow automation, and a web-based interface, LCRBenchmark provides a scalable foundation for future benchmarking studies and contributes to improving the reproducibility and accessibility of computational biology workflows.

Keywords: Low Complexity Regions, Protein Sequence Analysis, Benchmarking, Workflow Automation, Comparative tool evaluation, Bioinformatics

Contents

Certification	i
Acknowledgement	ii
Abstract	iii
Contents	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Protein Sequences and Low Complexity Regions	1
1.2 Computational Detection of LCRs	1
1.3 Need for Benchmarking	2
1.4 Motivation of the Present Work	2
2 Objectives	4
2.1 Primary Objective	4
2.2 Specific Objectives	4
2.2.1 Understanding the Benchmarking Framework	4
2.2.2 Installation and Execution of LCR Detection tools	4
2.2.3 Benchmarking Across Multiple Model Organisms	4
2.2.4 Standardization of Tool Outputs	5
2.2.5 Reproduction of the Benchmarking Pipeline	5
2.2.6 Workflow Automation	5
2.2.7 Development of a Web-Based Interface	5
2.2.8 Deployment of the Framework	5
3 Background and Literature Review	7
3.1 Low Complexity Region Detection	7
3.2 Existing Computational Tools	7
3.3 Existing Benchmarking Framework and Limitation	7
3.4 Scope of Present Work	8
4 Dataset Description	10
4.1 Selection of Reference Proteomes	10
4.2 Selected Organisms	10
4.3 Data Acquisition	11
4.4 Dataset Statistics	12

5	Computational Tools and Software Environment	13
5.1	Overview	13
5.2	LCR Detection Tools	13
5.3	Development Environment	13
6	Development of the Benchmarking Framework	14
6.1	Overview	14
6.2	Execution of LCR Detection Tools	14
6.3	Standardization of Tool Outputs	15
6.4	Development of Python Modules	16
6.5	Modular Design	16
7	Workflow Automation Using Snakemake	17
7.1	Introduction	17
7.2	Snakemake Workflow Design	17
7.3	Dependency Management	17
8	Development of LCRBenchmark	19
8.1	Introduction	19
8.2	Design Objectives	19
8.3	User Interface	20
8.4	Job Management	20
8.5	Additional Utilities	20
8.6	Full Figures and Results	22
8.7	Deployment	22
9	Testing, Validation and Representative Benchmark Results	25
9.1	Introduction	25
9.2	Validation of Framework Components	25
9.3	Workflow Validation	25
9.4	Evolution across multiple Organisms	26
10	Challenges Encountered and Lessons Learned	27
10.1	Introduction	27
10.2	Understanding Existing Workflow	27
10.3	Installation Of Multiple Command Line Tools	27
10.4	Standardizing Heterogeneous Outputs	28
10.5	Independent Development of Analysis Scripts	28
10.6	Workflow Automation	28
10.7	Web Application Development	28
10.8	Deployment Challenges	29
10.9	Lessons Learned	29
11	Future Scope	30
11.1	Introduction	30
11.2	Support for Additional Tools	30
11.3	Expansion to Additional Organisms	30
11.4	Performance Optimization	30
11.5	Enhanced Web Application	30

11.6 Cloud Deployment	31
12 Conclusion	32
12.1 Summary of Internship	32
12.2 Contributions	32
12.3 Final Outcome	32
Bibliography	34
A Detailed Proteome Metadata	35
A.1 Reference Proteome Profiles	35
B Python Scripts Developed	37
B.1 Shared Preprocessing Scripts	37
B.2 Figure 1: Baseline Properties (Length, Coverage, Composition) . . .	37
B.3 Figure 2: Consensus Regions & Peptide Motifs	38
B.4 Figure 3: Purity Metrics Across Tools	38
B.5 Figure 4: Inter-Tool Jaccard Overlaps	39
B.6 Figure 5: Sequence Mutation Profiling	39
B.7 Figure 6: Windowed Classification & TPR/FPR Bounds	39
B.8 Figure 7 & Documentation Generation	40
C Benchmark Results : Mouse	41

List of Figures

2.1	Project Workflow	6
4.1	Phylogenetic diversity	11
4.2	Data acquisition and downstream benchmarking execution.	11
6.1	Command Sheet : LCR Tools	15
7.1	DAG Graph	18
8.1	Homepage	19
8.2	Sample Data Files for Testing	20
8.3	Upload Input Files	21
8.4	Work Progress Page	22
8.5	Converter : Output of Tools to 1 Based BED	23
8.6	Result : Figures and Intermediate Files available	23
C.1	LCR Count across length bins	41
C.2	Coverage Heatmap.	42
C.3	Count Distribution.	43
C.4	Amino Acid Composition.	44
C.5	Shanon Entropy Boxplot.	45
C.6	Consensus regions and structural peptide motif variations	46
C.7	Shanon Entropy Distribution across Consensus levels.	47
C.8	Compositional Purity Across Consensus Levels.	48
C.9	Purity metrics distribution across individual tools.	49
C.10	Jaccard Similarity Matrix.	50
C.11	Amino Acid Composition Vs. Mutation Percentage in LCRs.	51
C.12	Gene Length Categories	52
C.13	No. of LCRs detected per gene.	53
C.14	LCR coverage percentage of the total gene length.	54
C.15	LCR-to-gene entropy ratio	55
C.16	Summary of benchmarking results across all evaluated LCR detection methods.	56

List of Tables

3.1	Overview of Sequence Analysis Tools and Detection Principles . . .	8
4.1	Model Organism Proteome Statistics from UniProt	12
5.1	Tool Configuration Matrix for Sequence Analysis	13
6.1	Overview of Software Modules and Analytical Tasks	14

1 Introduction

1.1 Protein Sequences and Low Complexity Regions

Proteins are among the most important biological molecules, performing a wide range of functions that are essential for the survival and normal functioning of living organisms. They participate in structural support, enzymatic reactions, cellular signaling, transport of molecules, immune responses, and regulation of gene expression. The function of a protein is largely determined by the sequence and arrangement of its constituent amino acids. Although proteins are often viewed as well-structured molecules, many proteins also contain regions that differ significantly from the rest of the sequence. One such class of regions is known as Low Complexity Regions (LCRs). Unlike typical protein regions that contain a diverse mixture of amino acids, LCRs are characterized by the repeated occurrence or overrepresentation of a small number of amino acids. These regions may consist of simple homopolymeric stretches, repetitive sequence motifs, or compositionally biased segments. For many years, LCRs were considered biologically unimportant because their repetitive nature made them difficult to study using conventional sequence analysis methods. However, research over the last two decades has shown that these regions play important roles in numerous biological processes. LCRs have been associated with transcriptional regulation, RNA binding, protein-protein interactions, phase separation, evolutionary adaptation, and several neurodegenerative disorders. Their widespread occurrence across different organisms has made them an important area of study in computational biology and proteomics. Despite their biological significance, identifying LCRs remains a computational challenge because there is no universally accepted definition of what constitutes a low-complexity region. Different computational methods define sequence complexity using different mathematical models, resulting in variations in the regions identified by different software tools.

1.2 Computational Detection of LCRs

Over the years, several computational methods have been developed to identify LCRs from protein sequences. Each tool approaches the problem from a different perspective and therefore detects different classes of low-complexity regions. Some methods, such as SEG, identify LCRs by measuring the local sequence complexity using entropy-based calculations. Other tools, including fLPS and fLPS 2.0, focus on identifying statistically significant compositional biases within protein sequences. Repeat-oriented methods such as T-REKS and XSTREAM specialize in detecting tandem repeats, while tools like AlcoR employ alignment-free statistical techniques to identify compositionally biased regions. LCRFinder analyzes sequence complexity using MATLAB-based algorithms, whereas Dotplot identifies repetitive regions through sequence self-similarity. Because these tools are based on different computational principles, they often produce different predictions even when analysing the same protein sequence. Some tools identify long compositionally biased regions,

while others preferentially detect short tandem repeats or highly repetitive motifs. Consequently, selecting an appropriate detection method depends on the biological question being investigated. This diversity of algorithms makes direct comparison between tools difficult, highlighting the need for systematic benchmarking.

1.3 Need for Benchmarking

Benchmarking is the process of evaluating multiple computational methods under a common framework using standardized datasets, identical evaluation criteria, and reproducible workflows. In computational biology, benchmarking provides an objective way to compare different algorithms, identify their strengths and limitations, and determine their suitability for specific biological applications. Recently, [Chatterjee and Vijay \[2026\]](#) proposed a comprehensive benchmarking framework for comparative evaluation of eight widely used LCR detection methods across the human proteome. Their framework compared the performance of different tools using multiple sequence-based and protein-based characteristics, including LCR length distribution, sequence coverage, amino acid composition, Shannon entropy, consensus analysis, sequence complexity, and several evaluation metrics. The study provided valuable insights into the similarities and differences between different LCR detection approaches and established a standardized methodology for comparative analysis. However, reproducing the complete benchmarking workflow required the execution of numerous independent scripts, multiple software tools, intermediate data processing steps, and manual coordination between different stages of the pipeline. Although the reference study provided some scripts through its GitHub repository, but many of the scripts were required to be made by the user, which made executing the workflow time-consuming and prone to user errors. Such challenges become even more significant when benchmarking needs to be performed on multiple organisms or when new computational tools need to be incorporated into the workflow. Therefore, there is a need for a reproducible, automated, and user-friendly framework that simplifies the benchmarking process while preserving the methodology proposed in the reference study

1.4 Motivation of the Present Work

This internship was carried out under the broader project titled "Benchmarking Computational Biology Tools Using Real and Simulated Data." The work presented in this report focuses specifically on automating the benchmarking framework for Low Complexity Region detection tools. The internship began with a detailed study of the published benchmarking framework and the computational methods used for LCR detection. To gain a practical understanding of the benchmarking process, all major LCR detection tools included in the reference study were installed and executed individually. These tools were subsequently applied to the reference proteomes of seven model organisms: *Arabidopsis thaliana*, *Caenorhabditis elegans*, *Danio rerio*, *Drosophila melanogaster*, *Mus musculus*, *Saccharomyces cerevisiae*, and *Escherichia coli*. Their outputs, originally generated in different file formats, were standardized into a common 1-based BED format using custom Python scripts. Once the complete manual workflow had been understood and reproduced, the focus

shifted towards improving its reproducibility and usability. The individual processing steps were integrated into a fully automated workflow using Snakemake, allowing the entire benchmarking pipeline to be executed through a single workflow definition while automatically managing dependencies and intermediate files. All downstream analyses, statistical processing, figure generation, and report preparation were implemented using Python, creating a unified computational environment. To further improve accessibility, a web application was developed using the Flask framework. The application enables users to upload proteome sequences and benchmark files, execute the complete pipeline through a graphical interface, monitor job progress in real time, and download figures, intermediate results, and automatically generated reports. Finally, the application was containerized using Docker and deployed on Render, making the benchmarking framework accessible through a web browser without requiring users to manually install the complete software environment. The resulting framework transforms a largely manual benchmarking workflow into an automated, reproducible, and user-friendly computational platform that can be applied to multiple proteomes with minimal user intervention.

2 Objectives

2.1 Primary Objective

The primary objective of this internship was to understand, reproduce, and automate the benchmarking workflow for computational tools used in the detection of Low Complexity Regions (LCRs) in protein sequences. The specific aim was to simplify the execution of an existing benchmarking framework through workflow automation and software development. The internship focused not only on reproducing the computational analyses described in the reference study, but also on transforming the complete benchmarking process into a reproducible, scalable, and user-friendly platform that could be executed with minimal manual intervention.

2.2 Specific Objectives

To accomplish the primary objective, the internship was divided into several smaller tasks.

2.2.1 Understanding the Benchmarking Framework

The first objective was to thoroughly study the published benchmarking framework for Low Complexity Region detection. This involved understanding the biological significance of LCRs, the computational principles behind different detection tools, the evaluation metrics used in the benchmark, and the overall sequence of analyses required to reproduce the published results.

2.2.2 Installation and Execution of LCR Detection tools

The next objective was to install, configure, and execute multiple LCR detection tools on a local computing environment. Each software package required a different installation procedure and produced outputs in its own native format. Running these tools independently provided practical experience with their execution, parameter settings, input requirements, and output structures. The tools included: SEG fLPS fLPS 2.0 AlcoR T-REKS XSTREAM LCRFinder Dotplot ([Lee et al. \[2022\]](#))

2.2.3 Benchmarking Across Multiple Model Organisms

Rather than restricting the workflow to the human proteome used in the reference study, another objective was to evaluate the benchmarking workflow on multiple model organisms representing different evolutionary groups. This provided an opportunity to validate the workflow on proteomes with varying sizes and biological complexity.

2.2.4 Standardization of Tool Outputs

Since every LCR detection tool generates its results in a different output format, direct comparison between tools is difficult. Therefore, one of the major objectives of this internship was to develop custom Python programs capable of converting these heterogeneous outputs into a common 1-based BED format. This standardized representation enabled all downstream analyses to operate on a single consistent data format, simplifying comparison between different computational methods.

2.2.5 Reproduction of the Benchmarking Pipeline

After generating standardized outputs, the next objective was to reproduce the computational analyses presented in the reference benchmark. This involved implementing the required processing steps, generating intermediate datasets, calculating sequence statistics, and reproducing the figures used for comparative evaluation of LCR detection tools. Although the reference implementation contained scripts developed using multiple programming languages, all analyses performed during this internship were implemented using Python, creating a unified computational workflow.

2.2.6 Workflow Automation

Executing every script manually required considerable effort and careful management of intermediate files. Another important objective was therefore to automate the entire benchmarking pipeline. The workflow was reconstructed using Snakemake, allowing every computational step to be represented as an independent rule with clearly defined inputs and outputs. This automation reduced manual intervention, improved reproducibility, and simplified the execution of complex multi-stage analyses.

2.2.7 Development of a Web-Based Interface

To make the benchmarking framework accessible to users without requiring command-line experience, another objective was to design and implement a graphical web interface. A web application was developed using the Flask framework, enabling users to upload datasets, execute the benchmarking workflow, monitor execution status, download generated figures, obtain intermediate files, and generate automated benchmark reports through a browser-based interface.

2.2.8 Deployment of the Framework

The final objective was to package the complete application for deployment. The benchmarking framework was containerized using Docker and deployed on the Render cloud platform, allowing the complete workflow to be executed through a publicly accessible web application without requiring users to install the underlying software dependencies.

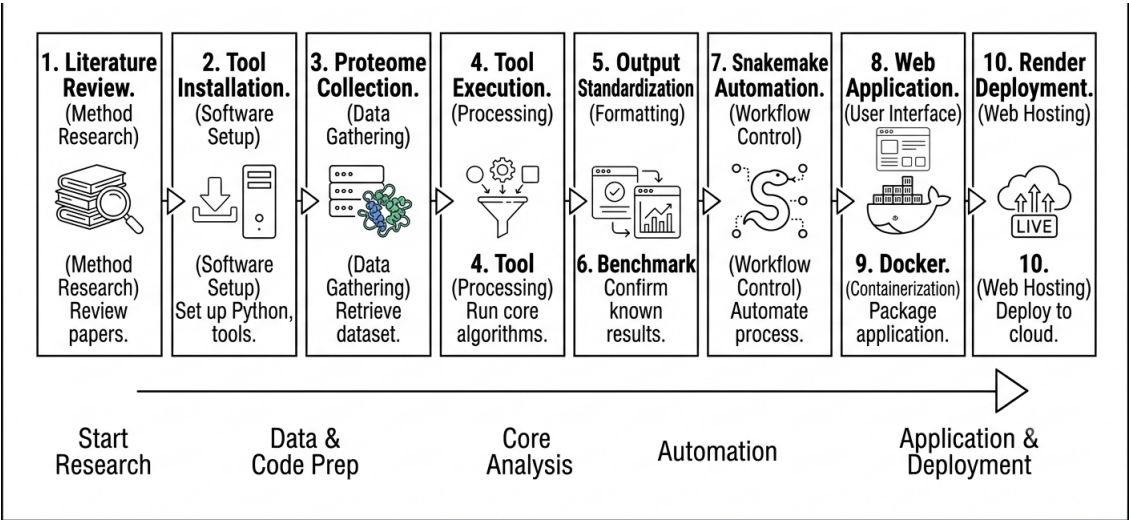


Figure 2.1: Project Workflow

3 Background and Literature Review

3.1 Low Complexity Region Detection

The identification of Low Complexity Regions has been an active area of research for several decades. As protein sequence databases expanded and whole-proteome analyses became feasible, computational methods were developed to automatically detect regions enriched with repetitive or compositionally biased amino acids. Unlike sequence alignment problems, LCR detection does not have a universally accepted definition. Different algorithms identify these regions using different mathematical and statistical approaches. As a result, two tools analysing the same protein sequence may predict different regions or assign different boundaries to the same LCR. This diversity has led to the development of several independent LCR detection tools, each designed with a specific computational strategy and optimized for particular types of sequence patterns.

3.2 Existing Computational Tools

A variety of computational tools have been developed for identifying Low Complexity Regions in protein sequences. Although they share the common objective of detecting regions with reduced sequence complexity, their underlying algorithms differ considerably. SEG [Wootton and Federhen \[1993\]](#) identifies regions of low sequence complexity using local sequence entropy and has been widely used for masking protein sequences prior to similarity searches. fLPS [Harrison \[2017\]](#) and fLPS 2.0 [Harrison \[2021\]](#) detect statistically significant compositional biases, making them particularly useful for identifying amino acid enrichment beyond simple tandem repeats. AlcoR [Silva et al. \[2023\]](#) employs alignment-free statistical models to detect compositionally biased regions based on sequence characteristics rather than sequence alignment. T-REKS [Jorda and Kajava \[2009\]](#) and XSTREAM [Newman and Cooper \[2007\]](#) are repeat-detection tools that identify tandemly repeated sequence motifs and repetitive amino acid patterns. LCRFinder is a MATLAB-based tool that detects low complexity regions using a sequence complexity algorithm implemented specifically for protein analysis. Unlike the above tools, Dotplot was not executed locally during this internship. Instead, the benchmark incorporated publicly available Dotplot predictions generated in the study by [Lee et al. \[2022\]](#), allowing its results to be included in the comparative evaluation without reimplementing the original methodology. Since each tool produces different output formats and follows different detection principles, directly comparing their predictions requires a common evaluation framework.

3.3 Existing Benchmarking Framework and Limitation

Among the available studies comparing LCR detection methods, the benchmarking framework proposed by [Chatterjee and Vijay \[2026\]](#) provided one of the most com-

Table 3.1: Overview of Sequence Analysis Tools and Detection Principles

Tool	Detection Principle	Output Format
SEG	Entropy-based	Masked FASTA
FLPS	Composition bias	Text
FLPS2	Composition bias	Text
AlcoR	Alignment-free statistics	Masked FASTA
T-REKS	Tandem repeats	TSV
XSTREAM	Repeat detection	HTML/Text
LCRFinder	MATLAB-based complexity	BED (processed)
Dotplot	Self-similarity	BED (reference dataset)

prehensive comparative evaluations of LCR detection tools. Rather than comparing tools using a single performance metric, the study evaluated their predictions from multiple perspectives. These included the number and length of detected LCRs, amino acid composition, sequence coverage, Shannon entropy, consensus among tools, sequence diversity, and protein-level characteristics. The benchmark also introduced a reference-based evaluation strategy in which predicted regions were classified according to predefined sequence composition criteria, enabling the calculation of performance metrics for comparative analysis. The classification approach used for the evaluation presented later in this report follows the methodology described in this benchmark. An important contribution of the study was that it demonstrated that no single LCR detection tool consistently outperformed all others. Instead, different algorithms exhibited strengths for different classes of low-complexity sequences, emphasizing the importance of selecting tools based on the intended biological application.

Manual workflows present several practical challenges: Execution requires familiarity with multiple software packages and file formats. Intermediate files must be managed manually. The absence of workflow management makes large-scale analyses more difficult to reproduce consistently. These limitations provided the motivation for developing a more structured and automated implementation.

3.4 Scope of Present Work

Building upon the existing benchmarking methodology, the work carried out during this internship focused on improving its usability, reproducibility, and accessibility without modifying the underlying evaluation strategy. The benchmarking pipeline was reconstructed as an automated workflow using Snakemake, allowing all processing steps to be executed in the correct order while automatically managing intermediate files and task dependencies. Custom Python programs were developed to standardize tool outputs, perform downstream analyses, and generate quality figures and reports within a single programming environment. To further improve accessibility, the complete workflow was integrated into a Flask-based web application and packaged using Docker for deployment on the Render cloud platform. While the deployment demonstrates the portability and accessibility of the framework, execution of the complete benchmarking pipeline on the free hosting tier is constrained by the available system resources (512 MB RAM). As several stages

of the workflow are computationally intensive, large benchmarking jobs exceed the memory limitations of the free instance. Nevertheless, the deployment validates the application's architecture and provides a foundation for future deployment on higher-resource infrastructure.

4 Dataset Description

4.1 Selection of Reference Proteomes

To evaluate the benchmarking framework across diverse biological systems, seven publicly available reference proteomes were selected from the UniProt Proteomes database. A reference proteome represents a curated set of protein sequences that provides comprehensive coverage of the protein-coding genes for an organism and is commonly used for comparative proteomics and computational analyses.

The selected organisms span multiple branches of the tree of life, including plants, fungi, invertebrates, vertebrates, and bacteria.

For each organism, the "one protein sequence per gene" FASTA dataset was downloaded to avoid redundancy arising from multiple protein isoforms. Using a single representative sequence for each gene ensures consistency across organisms and simplifies comparative analyses.

4.2 Selected Organisms

The benchmark was evaluated using the following seven model organisms:

***Arabidopsis thaliana*:** A widely studied flowering plant that serves as the primary model organism in plant biology. It was selected to represent plant proteomes within the benchmarking dataset.

***Caenorhabditis elegans*:** A free-living nematode extensively used in developmental biology and genetics. Its well-annotated proteome makes it suitable for computational benchmarking studies.

***Danio rerio*:** The zebrafish is an important vertebrate model organism widely used in developmental biology, genetics, and disease research. It provides a large vertebrate proteome for comparative evaluation.

***Drosophila melanogaster*:** The fruit fly has long served as a model organism for genetics and evolutionary biology. Its extensively annotated proteome has been widely used for computational sequence analysis.

***Mus musculus*:** The laboratory mouse is one of the most important mammalian model organisms and is frequently used in biomedical research. Its proteome enables evaluation of LCR (Low Complexity Region) detection methods in higher vertebrates.

***Saccharomyces cerevisiae*:** Baker's yeast is one of the best-characterized eukaryotic organisms and is widely used in molecular biology. Despite its relatively small proteome, it provides an important reference for fungal proteins.

***Escherichia coli* K-12:** One of the best-studied bacterial organisms. Including a bacterial proteome allows comparison between prokaryotic and eukaryotic sequence characteristics within the benchmarking framework.

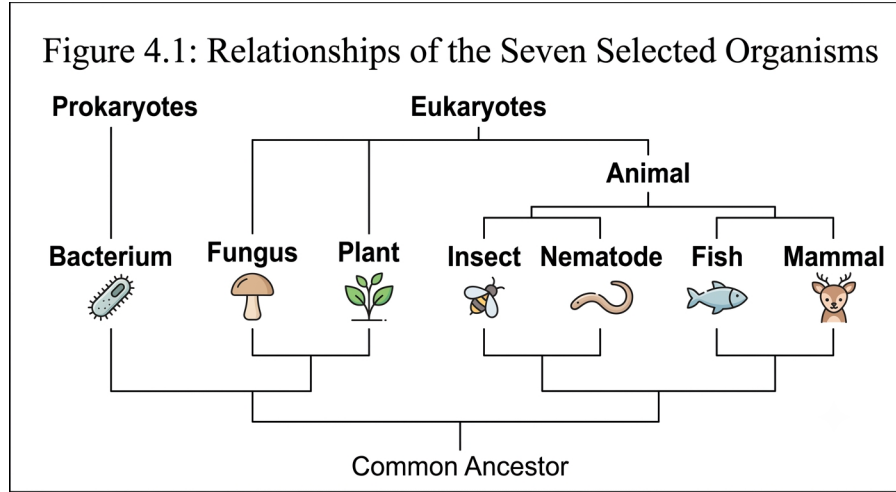


Figure 4.1: Phylogenetic diversity

4.3 Data Acquisition

All proteome datasets were downloaded directly from the UniProt Proteomes database in FASTA format. The datasets correspond to the reference proteomes available at the time of download and contain one representative protein sequence for each annotated gene.

The downloaded FASTA files served as the primary input for all LCR detection tools evaluated. Each tool processed the same input proteome for a given organism, ensuring that differences observed in downstream analyses reflected variations in the detection algorithms rather than differences in the underlying sequence data.

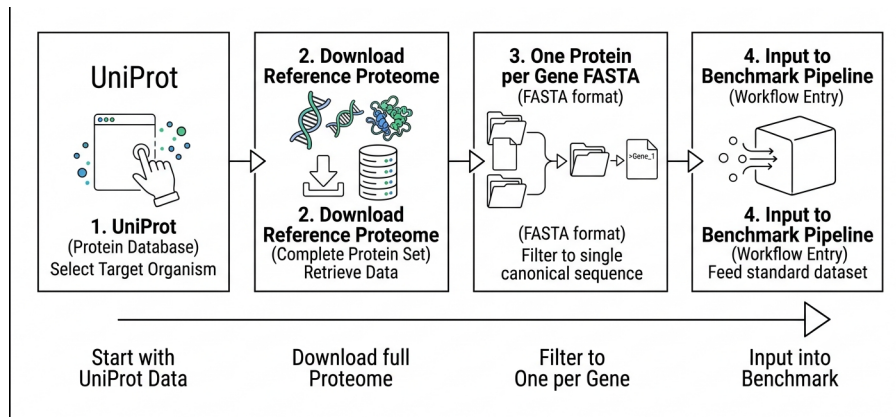


Figure 4.2: Data acquisition and downstream benchmarking execution.

4.4 Dataset Statistics

The quantitative characteristics of the retrieved reference proteomes vary significantly across the selected taxonomic groups. A detailed summary of the dataset metrics, including UniProt identifiers, total entries, gene counts, and NCBI Taxonomy IDs, is provided in Table 4.1.

Table 4.1: Model Organism Proteome Statistics from UniProt

Organism	Proteome ID	UniProt Entries	Gene Count	Taxon ID
<i>Arabidopsis thaliana</i>	UP000006548	39,273	27,496	3702
<i>Caenorhabditis elegans</i>	UP000001940	26,629	19,789	6239
<i>Danio rerio</i>	UP000000437	68,855	26,677	7955
<i>Drosophila melanogaster</i>	UP000000803	21,953	13,817	7227
<i>Mus musculus</i>	UP000000589	54,857	21,853	10090
<i>Saccharomyces cerevisiae</i>	UP000002311	6,067	6,066	559292
<i>Escherichia coli</i> K-12	UP000000625	4,403	4,403	83333

5 Computational Tools and Software Environment

5.1 Overview

The benchmarking framework evaluated during this internship consists of multiple computational tools, each designed to identify Low Complexity Regions (LCRs) using a different computational strategy. Since no single algorithm is capable of detecting every type of low-complexity sequence, benchmarking requires evaluating multiple methods under identical conditions. Each tool accepts protein sequences in FASTA format but differs in the algorithms used for detection, supported parameters, and output formats. Consequently, understanding the characteristics of each tool was an essential prerequisite before designing an automated benchmarking workflow. In addition to the LCR detection tools, several supporting software packages were used for data preprocessing, workflow automation, statistical analysis, visualization, and deployment. Together, these components form the complete computational environment used throughout this project.

5.2 LCR Detection Tools

Table 5.1: Tool Configuration Matrix for Sequence Analysis

Tool	Configurations Used
SEG	Default, Intermediate, Strict
fLPS	Default, Strict
fLPS 2.0	Default, Strict
AlcoR	Mode 1, Mode 2
XSTREAM	m1–m5*
T-REKS	ClustalW
LCRFinder	Default
Dotplot	Published dataset Lee et al. [2022]

5.3 Development Environment

The implementation was carried out primarily using Python, which served as the central programming language for the project. All custom scripts developed, including output conversion, preprocessing, statistical analysis, figure generation, report generation, and workflow integration, were implemented in Python. This improved maintainability, simplified debugging, and facilitated integration with the Snakemake workflow and Flask application.

6 Development of the Benchmarking Framework

6.1 Overview

After understanding the benchmarking methodology presented in the reference study [Chatterjee and Vijay \[2026\]](#) and familiarizing myself with the individual LCR detection tools. The complete benchmarking workflow was independently reconstructed by systematically studying the methodology described in the reference paper [Chatterjee and Vijay \[2026\]](#). Each stage of the analysis was then implemented in Python as modular scripts capable of performing individual computational tasks while remaining suitable for integration into an automated workflow.

Table 6.1: Overview of Software Modules and Analytical Tasks

Module	Major Tasks Performed
Input preprocessing	FASTA cleaning, BED preparation, directory setup
Figure 1 – Proteome stats	Sequence extraction, LCR length, entropy, coverage, amino acid composition, visualization
Figure 2 – Consensus	BEDTools intersection, consensus BED generation, motif analysis, entropy, purity
Figure 3 – Purity	Sequence extraction, purity calculation, visualization
Figure 4 – Similarity	Jaccard similarity calculation, overlap matrix, heatmap generation
Figure 5 – Complexity	Mutation metrics, heatmap generation, combined visualization
Figure 6 – Ref evaluation	Sliding windows, complexity calculation, LCR/HCR/CBR classification, confusion matrix, TPR/FPR calculation, plots
Figure 7 – Performance	Summary tables, comparative plots
Conversion scripts	Tool output conversion, helper functions, intermediate processing

6.2 Execution of LCR Detection Tools

Once the required software had been installed successfully, each LCR detection tool was executed independently on the seven reference proteomes described in Chapter 4. Since different tools produce fundamentally different output structures, no attempt was made to automate the workflow at this stage. Instead, the objective was to generate reliable outputs that could later be used for developing downstream analysis

scripts. The parameter values used for each software configuration are provided in Figure 6.1.

>_ **LCR TOOLS – COMMANDS CHEAT SHEET**

1 SEG (Default) <pre>segmasker \ -in arabidopsis.fasta \ -window 12 \ -locut 2.2 \ -hicut 2.5 \ -out arabidopsis_seg.fa \ -outfmt fasta</pre> 2 SEG (Intermediate) <pre>segmasker \ -in arabidopsis.fasta \ -window 15 \ -locut 1.9 \ -hicut 2.5 \ -out arabidopsis_seg_intermediate.fa \ -outfmt fasta</pre> 3 SEG (Strict) <pre>segmasker \ -in arabidopsis.fasta \ -window 15 \ -locut 1.5 \ -hicut 1.8 \ -out arabidopsis_seg_strict.fa \ -outfmt fasta</pre> 4 fLPS (Default) <pre>~/Lcr_Project/tools/flps/flps/flps/bin/linux/flps \ arabidopsis.fasta \ > flps_default.out</pre> 5 fLPS (Strict) <pre>~/Lcr_Project/tools/flps/flps/flps/bin/linux/flps \ -m 5 \ -M 25 \ -t 1e-5 \ arabidopsis.fasta \ > flps_strict.out</pre> 6 fLPS2 (Default) <pre>~/Lcr_Project/tools/flps2/flps2programs/src/flps2 \ arabidopsis.fasta \ > flps2_default.out</pre> 7 fLPS2 (Strict) <pre>~/Lcr_Project/tools/flps2/flps2programs/src/flps2 \ -m 5 \ -M 25 \ -t 1e-5 \ arabidopsis.fasta \ > flps2_strict.out</pre>	8 AlcoR (Mode 1) <pre>~/Lcr_Project/tools/alcor/alcor/bin/AlcoR mapper \ -v \ -m 5:20:0:0:10:0.9/3:10:0.9 \ -w 5 \ -k \ -o arabido_alcor_mode1_masked.fa \ arabidopsis.fasta</pre> 9 AlcoR (Mode 2) <pre>~/Lcr_Project/tools/alcor/alcor/bin/AlcoR mapper \ -v \ -m 5:10:0:0:10:0.9/1:1:0.9 \ -w 5 \ -k \ -o arabido_alcor_mode2_masked.fa \ Arabidopsis.fasta</pre> 10 XSTREAM m1 <pre>java -jar ~/Lcr_Project/tools/XSTREAM/xstream.jar \ arabidopsis.fasta \ -m1 \ -a_m1</pre> m2 <pre>java -jar ~/Lcr_Project/tools/XSTREAM/xstream.jar \ arabidopsis.fasta \ -m2 \ -a_m2</pre> m3 <pre>java -jar ~/Lcr_Project/tools/XSTREAM/xstream.jar \ arabidopsis.fasta \ -m3 \ -a_m3</pre> m4 <pre>java -jar ~/Lcr_Project/tools/XSTREAM/xstream.jar \ arabidopsis.fasta \ -m4 \ -a_m4</pre> m5 <pre>java -jar ~/Lcr_Project/tools/XSTREAM/xstream.jar \ arabidopsis.fasta \ -m5 \ -a_m5</pre> (Change the value of -m from 1 to 5 for respective modes) 11 T-REKS (ClustalW) <pre>java -Xmx4G \ -jar ~/Lcr_Project/tools/treks-hpc/target/ T-ReksHPC_0.1-SNAPSHOT.jar \ -f arabidopsis.fasta \ -t arabido_treks_clustalw.tsv \ -a arabido_treks_clustalw.aln \ -c /usr/bin/clustalw</pre>
--	---

SUMMARY	● SEG : 3 (Default, Intermediate, Strict) ● fLPS : 2 (Default, Strict) ● fLPS2 : 2 (Default, Strict) ● AlcoR : 2 (Mode 1, Mode 2) ● XSTREAM : 5 (m1, m2, m3, m4, m5) ● T-REKS : 1 (ClustalW)	NOTE Replace arabidopsis.fasta with your input FASTA file path as needed. Ensure all tools are installed and paths are correct before running these commands.
Total Tools : 8 Total Configurations : 15		

Figure 6.1: Command Sheet : LCR Tools

6.3 Standardization of Tool Outputs

One of the first practical challenges encountered during the project was the lack of a common output format among the different LCR detection tools. For example: SEG generated masked FASTA sequences. fLPS and fLPS2 produced plain tabular annotations. AlcoR generated masked protein sequences. T-REKS produced tab-separated repeat annotations. XSTREAM generated HTML and tabular outputs. LCRFinder produced MATLAB-files for each sequence separately, which were

combined and converted to 1 based BED format using user defined MATLAB functions. These heterogeneous outputs could not be compared directly. To address this issue, dedicated Python programs were developed for each software package to extract the predicted Low Complexity Regions and convert them into a standardized 1-based BED format. Using a common coordinate system simplified all downstream analyses, as every subsequent script operated on the same input format regardless of the software used to generate the predictions.

6.4 Development of Python Modules

After obtaining standardized BED files, the next objective was to reproduce the computational analyses presented in the reference benchmark. Each script was developed after studying the methodology described in the publication and understanding the expected outputs for each analysis. Description of each script can be found in Appendix B. The resulting scripts were designed to perform individual tasks such as sequence extraction, entropy calculation, purity estimation, consensus generation, overlap analysis, reference-based classification, statistical summarization, and figure generation. Since every component was implemented within the same programming environment, the overall workflow became easier to maintain, debug, and automate.

6.5 Modular Design

A key objective during development was to ensure that every stage of the benchmarking process remained independent and reusable. Instead of creating one large analysis program, the workflow was divided into multiple Python modules, each responsible for a single computational task. The output generated by one module served as the input for the next, allowing individual components to be tested, modified, or replaced without affecting the remainder of the workflow. This modular organization significantly simplified debugging and later enabled straightforward integration with the Snakemake workflow management system.

7 Workflow Automation Using Snakemake

7.1 Introduction

The benchmarking framework developed consists of multiple computational stages, beginning with the execution of Low Complexity Region (LCR) detection tools and progressing through data preprocessing, statistical analysis, figure generation, and report creation. Each stage depends on the successful completion of one or more previous stages, forming a complex sequence of interdependent computational tasks. Figure 7.1 shows the Directed Acyclic Graph (DAG) representing the complete workflow of the benchmarking pipeline, illustrating the sequence of tasks and their dependencies. Executing these tasks manually is both time-consuming and error-prone. A missing intermediate file, an incorrectly executed script, or running analyses in the wrong order can prevent successful completion of the benchmark. These challenges become increasingly significant when the workflow is executed repeatedly across multiple proteomes. To address these limitations, the complete benchmarking pipeline was automated using Snakemake, a workflow management system widely used in bioinformatics for building reproducible computational pipelines.

7.2 Snakemake Workflow Design

The complete benchmarking framework was implemented as a Snakemake workflow, where each computational stage was represented as an independent rule with explicitly defined input and output files. Rather than executing scripts sequentially through shell commands, Snakemake constructs a Figure 7.1 Directed Acyclic Graph (DAG) representing dependencies between all computational tasks. Based on this dependency graph, only the required rules are executed, while completed stages with unchanged outputs are automatically skipped. This dependency-driven execution improves efficiency by avoiding unnecessary recomputation and ensures that every analysis is performed in the correct order.

7.3 Dependency Management

One of the major advantages of Snakemake is its automatic dependency resolution. Every rule explicitly specifies the files it requires as input and the files it produces as output. Before executing a rule, Snakemake verifies that all required inputs are available. If an input file has not yet been generated, the workflow automatically executes the upstream rule responsible for creating that file. This mechanism eliminates the need for manually determining execution order and significantly reduces the possibility of user error.

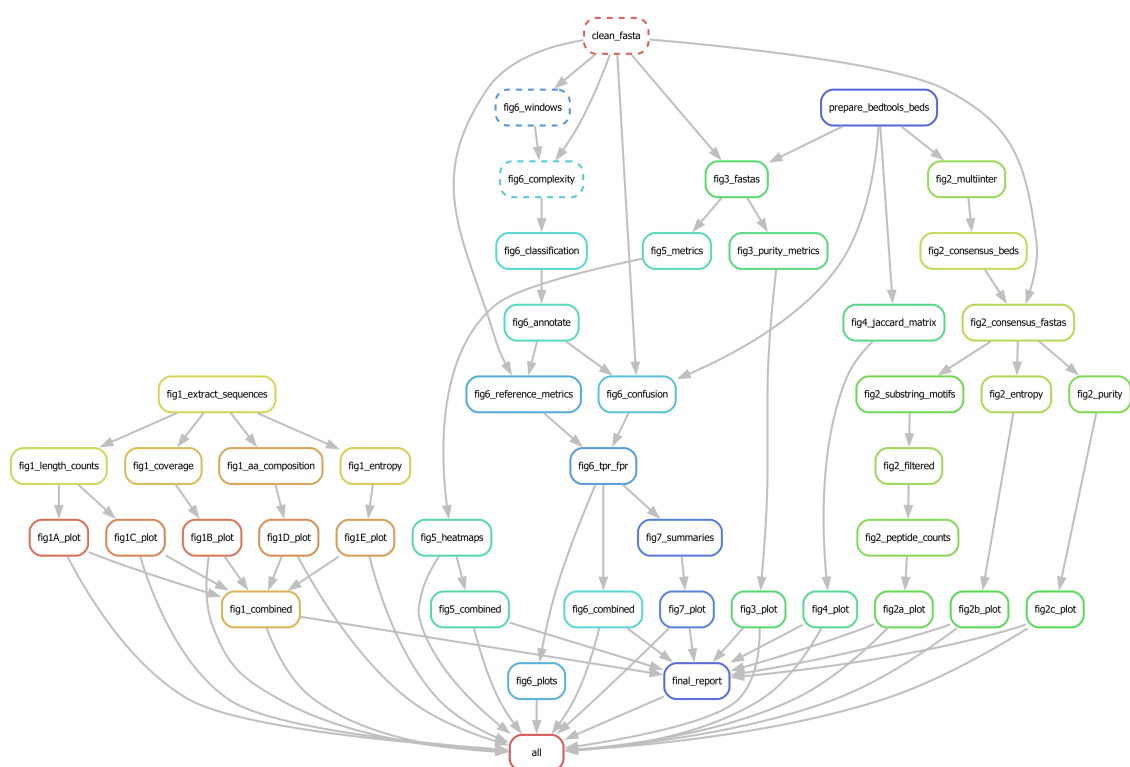


Figure 7.1: DAG Graph

8 Development of LCRBenchmark

8.1 Introduction

Although the Snakemake workflow automated the execution of the benchmarking pipeline, users were still required to interact with the command-line interface to provide inputs, configure parameters, execute workflows, and retrieve results. This approach is suitable for experienced users but can present a significant barrier for researchers with limited computational experience. To improve accessibility, a web application was developed using the Flask framework. The objective was to provide a graphical interface through which users could execute the complete benchmarking workflow without manually interacting with the underlying scripts or workflow management system. The web application serves as a bridge between the user and the computational pipeline, allowing complex analyses to be performed through a browser-based interface while preserving the reproducibility provided by the Snakemake workflow.

8.2 Design Objectives

The development of the web application was guided by the following objectives: Provide a simple graphical interface for executing the benchmarking workflow. Eliminate the need for command-line interaction. Allow users to upload input datasets through a web browser. Automate execution of the Snakemake workflow. Provide access to intermediate files and benchmark outputs, generate downloadable reports and visualizations. The Flask application is responsible for user interaction, while Snakemake manages workflow execution. The computational analyses remain encapsulated within the modular Python scripts developed.

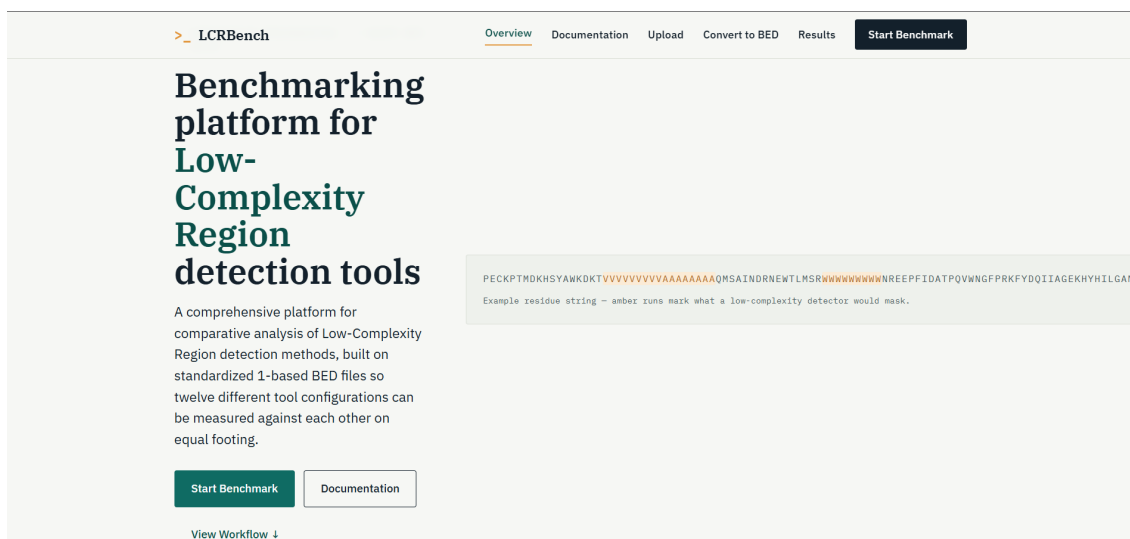


Figure 8.1: Homepage

8.3 User Interface

The application provides a browser-based interface that guides users through the complete benchmarking process. The workflow begins with dataset submission, where users upload the required input files. After validation of the uploaded files, the application creates a dedicated working directory for the submitted job and initiates execution of the Snakemake workflow. Throughout execution, the application continuously monitors workflow progress and presents the current status through the web interface. Once the analysis has completed successfully, users can access the generated figures, summary tables, intermediate outputs, and benchmark report directly from the browser.

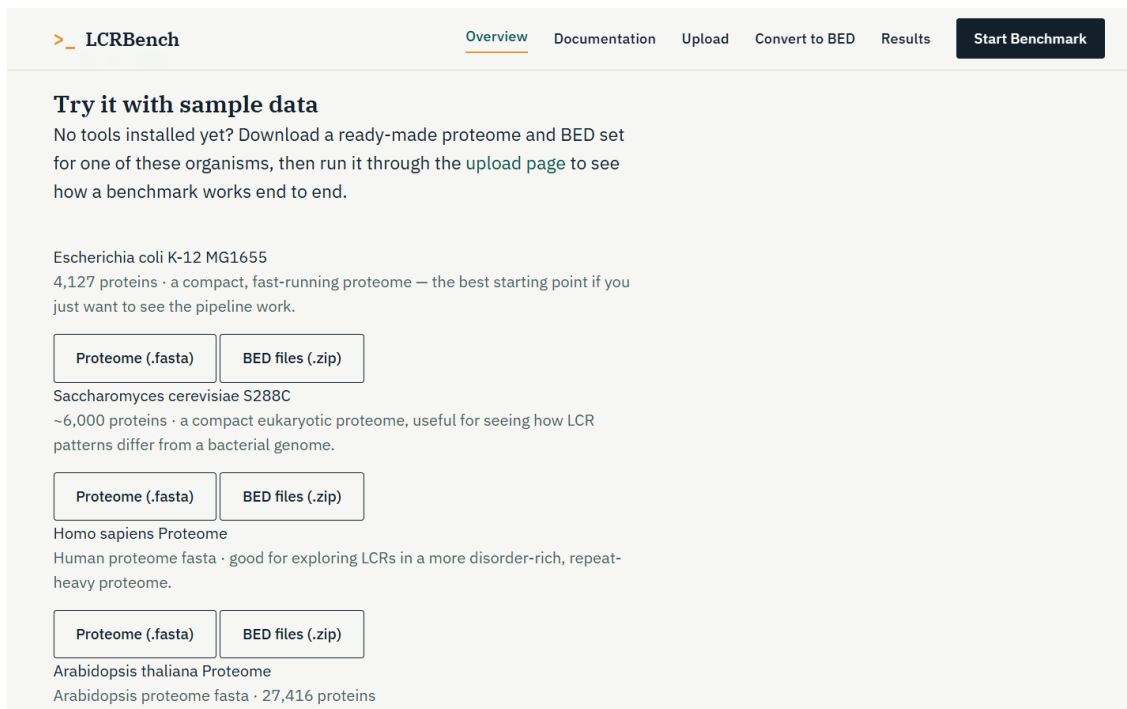


Figure 8.2: Sample Data Files for Testing

8.4 Job Management

Each benchmarking request submitted through the web interface is assigned an Job ID and the folder with this name contains the uploaded inputs, intermediate files, generated figures, and final reports. Maintaining separate execution directories provides several advantages: prevents conflicts between multiple analyses; preserves intermediate files for later inspection; simplifies debugging; enables users to download all generated outputs associated with a particular benchmark. This organization also facilitates future extension of the application to support concurrent users.

8.5 Additional Utilities

In addition to providing access to the benchmarking workflow, the application includes several utility functions that simplify preprocessing and result management.

7/13/26, 8:19 PM

Upload Benchmark Inputs — LCRBench

>_ LCRBench

06 · RUN A BENCHMARK

Upload your BED files

Upload standardized 1-based BED files from any of the twelve supported configurations, plus the source proteome. More configurations increase benchmarking quality — but the analysis runs with fewer.

Organism / sample tag

Used to name output files, e.g. `ecoli`. Required.

e.g. `ecoli`

Proteome (FASTA)

The source proteome your BED files were generated from. Required.

Choose File

No file chosen

⬆

Drop BED files here, or click to browse

Accepts up to 12 files, following the naming convention below.

Configuration coverage

0 / 12

Required naming convention

Name the file with tool name and configuration

alcor_mode1_masked.bed

alcor_mode2_masked.bed

flps_default.bed

flps_strict.bed

flps2_default.bed

flps2_strict.bed

seg_default.bed

seg_intermediate.bed

seg_strict.bed

treks_clustalw.bed

xstream_m1.bed

lcrfinder.bed

<https://project-lcr.onrender.com>

1/2

Figure 8.3: Upload Input Files

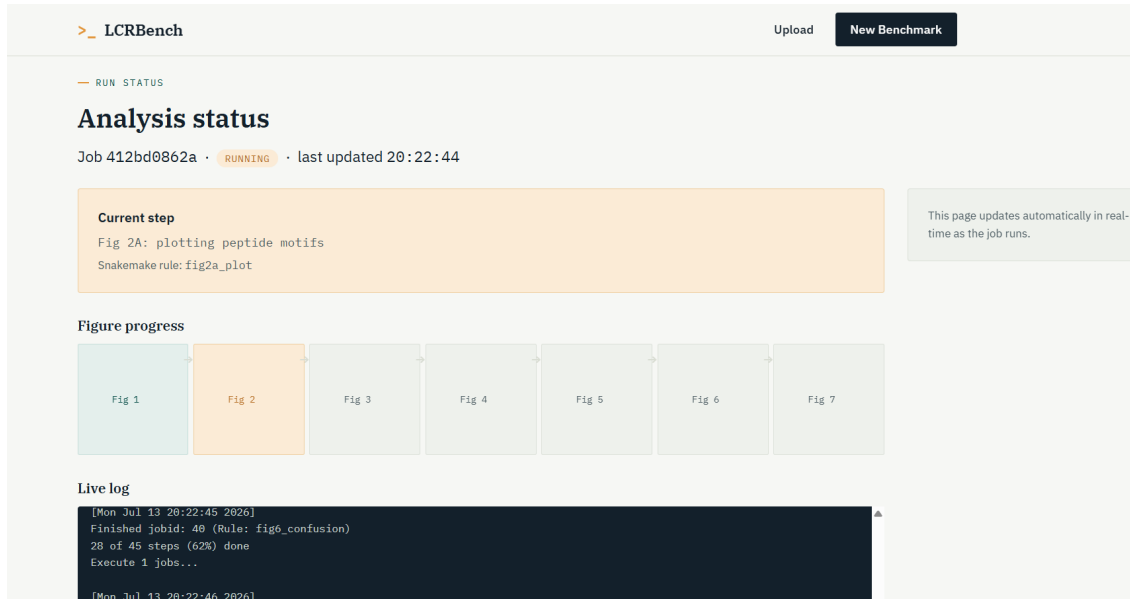


Figure 8.4: Work Progress Page

These utilities assist users in preparing benchmark inputs, converting tool outputs into compatible formats, providing the sample data for analysis (considering user just wants to use this website to learn about different LCR benchmarking metrics and doesn't intend to download the tools in their system). Also besides all this, a word doc and the [GitHub repository](#) was attached so anyone interested in more details can refer to that. Providing these auxiliary features within the same application reduces the need for external preprocessing scripts and streamlines the overall benchmarking process.

8.6 Full Figures and Results

Following successful completion of the benchmarking workflow, the generated visualizations are compiled into a structured benchmark report with caption and figure explanation. This eliminates the need for manual collection of figures and enables users to obtain a comprehensive view of the benchmarking results immediately after execution. User can download the The benchmark report (Pdf with figures and captions), Individual Figures (PNGs), Zip file of intermediary files, Comprehensive Result Package (includes figures, intermediate data files, execution logs, and associated metadata) are compiled into a single unified archive for streamlined access and distribution.

8.7 Deployment

Although the [GitHub repository](#) is publicly available, anyone can clone it and run in their system. It has README file, which will help to run the pipeline on any machine. To simplify software distribution and ensure reproducibility across different computing environments, the complete web application was containerized using Docker. Containerization packages the application together with all required software dependencies, ensuring consistent behaviour across different operating systems. The

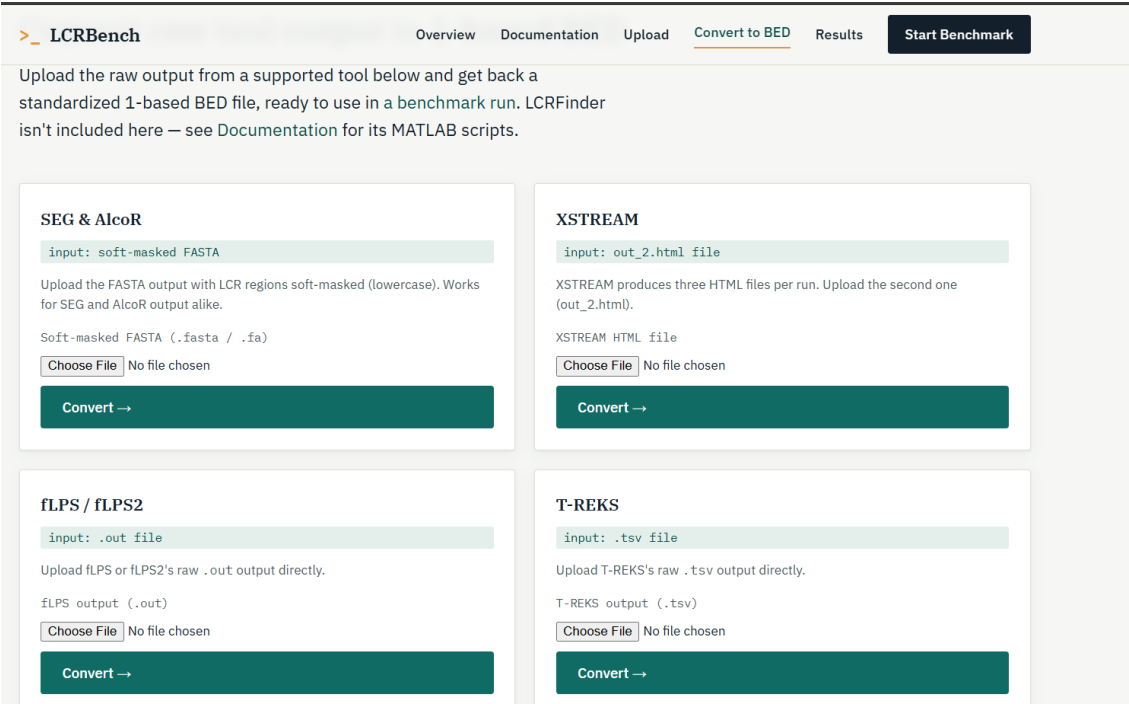


Figure 8.5: Converter : Output of Tools to 1 Based BED

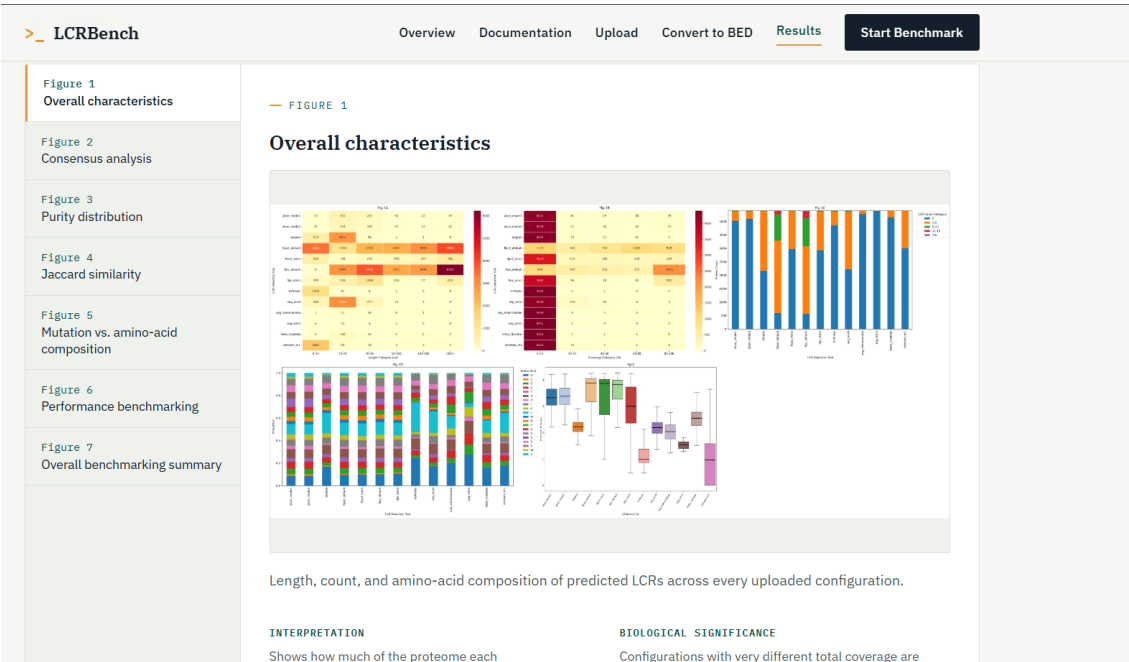


Figure 8.6: Result : Figures and Intermediate Files available

Dockerized application was subsequently deployed on the Render cloud platform, making the benchmarking framework accessible through a web browser. Although the deployment successfully demonstrates the portability of the framework, execution of the complete benchmarking workflow on the free hosting tier is constrained by the available system resources. The free Render instance provides 512 MB of RAM, which is insufficient for several computationally intensive stages of the pipeline when processing large proteomes. The source code of LCRBenchmark is maintained in a public GitHub repository to facilitate reproducibility, version control, and future development. A deployed version of the application is also available through the Render cloud platform for demonstration purposes.

GitHub Repository: https://github.com/Harshit-singh15/LCR_Project

Web Application: <https://project-lcr.onrender.com/>

9 Testing, Validation and Representative Benchmark Results

9.1 Introduction

The development of an automated benchmarking framework requires validation at multiple stages to ensure that each component functions correctly and that the complete pipeline produces reliable outputs. Validation was performed progressively as individual modules were developed and later after integration into the complete workflow. Rather than evaluating only the final outputs, testing was carried out throughout the development process. Each computational module, conversion script, workflow stage, and web application component was verified independently before becoming part of the complete benchmarking framework. This incremental validation approach simplified debugging and improved the overall reliability of the system.

9.2 Validation of Framework Components

The validation process began with the successful execution of all LCR detection tools on the selected reference proteomes, confirming that the input FASTA files were processed correctly, the specified parameter configurations were applied, and the expected output files were generated without errors.

Since each tool produced predictions in a different output format, the Python conversion utilities were subsequently validated to ensure accurate transformation of these outputs into a standardized 1-based BED format. Particular attention was given to preserving protein identifiers, maintaining correct LCR boundaries, and ensuring coordinate consistency across all converted files. This standardized representation enabled uniform processing by all downstream analysis modules.

Following output standardization, Python scripts were used to reproduce the benchmarking analyses described in the [Chatterjee and Vijay \[2026\]](#). Intermediate outputs, statistical summaries, and generated visualizations were systematically verified against the methodology presented in the reference study to ensure consistency of implementation.

The successful validation of tool execution, output standardization, and benchmark reproduction established the reliability of the complete benchmarking pipeline and provided confidence in the correctness of the automated workflow.

9.3 Workflow Validation

Following the successful validation of the individual analysis scripts, the complete Snakemake workflow was tested. Validation focused on verifying that: workflow dependencies were resolved correctly; all rules executed in the intended sequence; intermediate files were generated automatically; failed executions could be resumed successfully; the complete benchmarking pipeline produced identical outputs across

repeated executions. The dependency-driven execution provided by Snakemake eliminated manual coordination between processing stages while improving reproducibility.

9.4 Evolution across multiple Organisms

To assess the applicability of the framework beyond a single proteome, the complete benchmarking workflow was executed using reference proteomes from seven model organisms representing different taxonomic groups. Successful execution across organisms with substantially different proteome sizes demonstrated that the developed framework is independent of organism-specific characteristics and can be applied to diverse biological datasets with minimal modification.

10 Challenges Encountered and Lessons Learned

10.1 Introduction

Developing an automated benchmarking framework involved considerably more than executing existing computational tools. Throughout the internship, several technical challenges were encountered during software installation, workflow development, data standardization, automation, and deployment. Addressing these challenges required understanding the computational methodology described in the study of [Chatterjee and Vijay \[2026\]](#), independently developing analysis scripts, and integrating multiple software components into a unified framework. The solutions developed during this process not only enabled successful completion of the project but also provided valuable experience in computational biology, workflow engineering, and software development.

10.2 Understanding Existing Workflow

One of the earliest challenges was understanding the complete benchmarking methodology described in the study. Although the study clearly described the biological analyses and presented the final benchmarking results, translating the methodology into an executable computational workflow required careful interpretation of multiple analysis steps. Understanding how individual figures were generated, identifying intermediate processing stages, and determining the relationships between different analyses required detailed examination of both the publication and the accompanying resources. This process provided valuable insight into the importance of reproducible computational workflows and highlighted the need for better organization of complex bioinformatics pipelines.

10.3 Installation Of Multiple Command Line Tools

The benchmarking framework incorporates several independent LCR detection tools developed using different programming languages and software environments. Installing these tools required resolving software dependencies, configuring execution environments, and adapting older software packages to the available computing platform. Some tools required Java, others depended on MATLAB, while additional software packages required external utilities such as ClustalW or BLAST+. Successfully integrating these diverse software packages into a single computational environment formed an essential step in the development of the benchmarking framework.

10.4 Standardizing Heterogeneous Outputs

Each LCR detection tool generates predictions using its own output format and coordinate representation. Some produced masked FASTA sequences, others generated tabular annotations, HTML reports, or MATLAB-specific outputs. Since these outputs could not be compared directly, dedicated Python programs were developed to extract the relevant information and convert every prediction into a standardized 1-based BED format. These conversion utilities required careful interpretation of each tool's output while ensuring that sequence coordinates and protein identifiers were preserved accurately.

10.5 Independent Development of Analysis Scripts

Although the study [Chatterjee and Vijay \[2026\]](#) provided supporting scripts through a public GitHub repository, reproducing the complete benchmarking workflow directly from these resources proved difficult because the execution sequence, intermediate processing steps, and dependencies were not explicitly organized into a complete workflow. Rather than adapting the existing implementation, the downstream analysis pipeline was independently reconstructed by studying the methodology presented in the publication. Approximately 45 modular Python scripts were developed to perform data preprocessing, statistical analysis, figure generation, report creation, and intermediate processing. This modular approach improved readability, debugging, and later enabled straightforward workflow automation.

10.6 Workflow Automation

Automating the benchmarking process presented its own set of challenges. The pipeline contains numerous dependent computational stages shown in [Figure 7.1](#), making manual execution difficult to maintain. Designing a workflow that correctly managed dependencies while remaining modular required careful organization of inputs, outputs, and intermediate files. Implementing the framework in Snakemake addressed these challenges by providing automatic dependency management, reproducible execution, and the ability to resume interrupted analyses without restarting the complete pipeline.

10.7 Web Application Development

Transforming a command-line workflow into a browser-based application required integrating multiple software components into a unified system. The web application had to manage user uploads, organize project directories, invoke the Snakemake workflow, monitor execution status, and provide access to generated reports and figures. Designing the application in a modular manner ensured that improvements to the computational pipeline could be incorporated without requiring major modifications to the user interface. The resulting software, LCRBenchmark, provides an accessible interface for executing complex benchmarking analyses while preserving the reproducibility of the underlying workflow.

10.8 Deployment Challenges

Following completion of the web application, the framework was containerized using Docker and deployed on the Render cloud platform. While the deployment successfully demonstrated the portability of the software, practical limitations were encountered due to the hardware resources available under the free hosting tier. The available 512 MB RAM was insufficient for executing computationally intensive analyses involving large proteomes, preventing successful completion of the complete benchmarking workflow through the deployed instance.

10.9 Lessons Learned

This internship provided experience extending beyond computational biology and exposed several important aspects of scientific software development. The project emphasized the importance of reproducible research, modular software design, workflow automation, and standardized data formats in large-scale computational analyses. It also demonstrated how independent software components can be integrated into a complete analytical framework through workflow management and web technologies. It also established the fact that working on linux system is better than windows because it provides many bioinformatics tool directly, which we need to download separately in windows. From a technical perspective, the internship provided practical experience with Python-based scientific programming, Snakemake workflow development, Docker containerization, Flask web application development, cloud deployment, and software engineering practices applicable to computational biology.

11 Future Scope

11.1 Introduction

The benchmarking framework developed during this internship establishes a reproducible platform for the comparative evaluation of Low Complexity Region (LCR) detection tools. While the current implementation successfully automates the complete benchmarking workflow and provides a user-friendly web interface, several opportunities remain for extending the framework in terms of functionality, scalability, and accessibility. The modular architecture adopted during development makes it possible to incorporate these enhancements without major modifications to the existing workflow.

11.2 Support for Additional Tools

The current implementation includes eight widely used LCR detection methods. As new algorithms become available, they can be incorporated into the benchmarking framework by developing the required output conversion utilities and integrating them into the existing Snakemake workflow. This would allow the framework to remain up-to-date with future developments in computational methods for LCR detection.

11.3 Expansion to Additional Organisms

Although the framework was validated using seven reference proteomes representing diverse taxonomic groups, it can readily be applied to additional organisms. Future studies may benchmark: larger eukaryotic proteomes, non-model organisms, pathogenic species, viral proteomes, custom user-provided datasets. Such analyses would broaden the applicability of the benchmarking framework.

11.4 Performance Optimization

Several stages of the benchmarking workflow are computationally intensive, particularly when processing large proteomes. Future work may focus on: reducing memory consumption, optimizing execution time, increasing parallelization, improving handling of very large datasets. Such optimizations would improve the scalability of the framework and enable efficient execution on resource-constrained systems.

11.5 Enhanced Web Application

The current version of LCRBenchmark provides a browser-based interface for executing the benchmarking workflow. Future versions may include additional features such as: user authentication, project history, execution notifications, interactive

visualization dashboards, parameter customization, online result comparison, downloadable publication-ready reports in multiple formats. These additions would further improve usability and accessibility.

11.6 Cloud Deployment

The application has been successfully containerized using Docker and deployed on the Render platform. However, execution of computationally intensive workflows is currently limited by the hardware resources available on the free hosting tier. Future deployment on dedicated servers or cloud platforms with higher computational resources would enable complete online execution of large benchmarking jobs while supporting multiple simultaneous users.

12 Conclusion

12.1 Summary of Internship

This internship was carried out under the project titled "Benchmarking Computational Biology Tools Using Real and Simulated Data," with the objective of understanding, reproducing, and automating the benchmarking of computational tools used for the detection of Low Complexity Regions (LCRs) in protein sequences. The work began with an in-depth study of the published benchmarking framework and the computational methods employed for LCR detection. This provided the foundation for understanding the characteristics of different detection algorithms, their output formats, and the sequence of analyses required to perform a comprehensive comparative evaluation. The selected LCR detection tools were successfully installed and executed on seven reference proteomes representing a diverse range of model organisms. A significant part of the internship involved addressing the practical challenges associated with reproducing an existing computational workflow. Since the outputs generated by different tools were incompatible with one another, dedicated Python programs were developed to convert them into a standardized 1-based BED format. This common representation enabled uniform downstream processing and simplified comparative analysis across all benchmarked methods.

12.2 Contributions

Approximately 45 modular Python scripts were developed to perform preprocessing, statistical analysis, benchmark evaluation, figure generation, and automated report creation. The modular organization of these scripts improved maintainability and established the foundation for workflow automation. To eliminate the need for manual execution, the complete benchmarking framework was reconstructed as a Snakemake workflow capable of automatically managing dependencies, intermediate files, and execution order. This transformation significantly improved the reproducibility and scalability of the benchmarking process while reducing manual intervention during repeated analyses. To further improve accessibility, the automated workflow was integrated into a Flask-based web application named LCRBenchmark. The application provides a graphical interface through which users can upload datasets, execute the benchmarking workflow, monitor analysis progress, and download generated reports and visualizations without requiring command-line interaction. The software was subsequently containerized using Docker and deployed on the Render cloud platform, demonstrating its portability and readiness for cloud-based execution.

12.3 Final Outcome

Although the free deployment environment imposes computational limitations for large-scale analyses, the successful implementation of the complete framework demon-

strates that complex benchmarking workflows can be transformed into reproducible, modular, and user-friendly software platforms. The architecture developed during this internship provides a strong foundation for future extensions, including support for additional LCR detection tools, larger biological datasets, and deployment on higher-performance computing infrastructure. Overall, the internship successfully achieved its objectives by progressing from understanding an existing benchmarking methodology to developing an automated computational framework that integrates data standardization, workflow management, web-based execution, and cloud deployment.

Bibliography

- Anirjeet Chatterjee and Nagarjun Vijay. A benchmarking framework for comparative evaluation of low-complexity region detection tools in the human proteome. *Scientific Reports*, 16:17498, 2026. doi: 10.1038/s41598-026-48432-3. <https://doi.org/10.1038/s41598-026-48432-3>.
- Paul M. Harrison. flps: Fast discovery of compositional biases for the protein universe. *BMC Bioinformatics*, 18:476, 2017. doi: 10.1186/s12859-017-1906-3. fLPS. Source code: <https://github.com/pmharrison/flps>.
- Paul M. Harrison. flps 2.0: Rapid annotation of compositionally-biased regions in biological sequences. *PeerJ*, 9:e12363, 2021. doi: 10.7717/peerj.12363. fLPS 2.0. Source code: <https://github.com/pmharrison/flps2>.
- Julien Jorda and Andrey V. Kajava. T-reks: identification of tandem repeats in sequences with a k-means based algorithm. *Bioinformatics*, 25(20):2632–2638, 2009. doi: 10.1093/bioinformatics/btp482. T-REKS. Tool homepage: <https://bioinfo.crbm.cnrs.fr/index.php?route=tools&tool=3>.
- Byron Lee, Nima Jaber-Lashkari, and Eliezer Calo. A unified view of low complexity regions (lcrs) across species. *eLife*, 11:e77058, 2022. doi: 10.7554/eLife.77058. <https://doi.org/10.7554/eLife.77058>.
- Aaron M. Newman and James B. Cooper. Xstream: A practical algorithm for identification and architecture modeling of tandem repeats in protein sequences. *BMC Bioinformatics*, 8:382, 2007. doi: 10.1186/1471-2105-8-382. XSTREAM. Tool homepage: <http://jimcooperlab.mcdb.ucsb.edu/xstream>.
- Jorge M. Silva, Weihong Qi, Armando J. Pinho, and Diogo Pratas. Alcor: alignment-free simulation, mapping, and visualization of low-complexity regions in biological data. *GigaScience*, 12:giad101, 2023. doi: 10.1093/gigascience/giad101. Alcor. Source code: <https://github.com/cobilab/alcor>.
- John C. Wootton and Scott Federhen. Statistics of local complexity in amino acid sequences and sequence databases. *Computers & Chemistry*, 17(2):149–163, 1993. doi: 10.1016/0097-8485(93)85006-X. SEG. Source code: <ftp://ftp.ncbi.nih.gov/pub/seg/seg/> (also bundled in the NCBI BLAST+ package as `segmasker`).

A Detailed Proteome Metadata

This appendix provides exhaustive reference metadata for the seven model organism proteomes retrieved from the UniProt Proteomes database. These records correspond to the exact releases utilized to execute the benchmarking pipeline, including specific laboratory strains and genome assembly accessions to ensure exact computational reproducibility.

A.1 Reference Proteome Profiles

The complete dataset configuration parameters for each organism are enumerated below:

Drosophila melanogaster (Fruit fly)

- **Proteome ID:** UP000000803
- **Status:** Reference proteome
- **Total UniProtKB Entries:** 21,953
- **Gene Count (Canonical FASTA):** 13,817
- **Taxonomy / Strain:** Berkeley
- **NCBI Taxon ID:** 7227
- **Genome Assembly & Annotation:** GCA_000001215.4 (ENA/EMBL)

Danio rerio (Zebrafish)

- **Proteome ID:** UP000000437
- **Status:** Reference proteome
- **Total UniProtKB Entries:** 68,855
- **Gene Count (Canonical FASTA):** 26,677
- **Taxonomy / Strain:** Tuebingen
- **NCBI Taxon ID:** 7955
- **Genome Assembly & Annotation:** GCF_049306965.1 (RefSeq)

Mus musculus (Mouse)

- **Proteome ID:** UP000000589
- **Status:** Reference proteome
- **Total UniProtKB Entries:** 54,857
- **Gene Count (Canonical FASTA):** 21,853
- **Taxonomy / Strain:** C57BL/6J
- **NCBI Taxon ID:** 10090

- **Genome Assembly & Annotation:** GCA_000001635.9 (Ensembl)

Caenorhabditis elegans

- **Proteome ID:** UP000001940
- **Status:** Reference proteome
- **Total UniProtKB Entries:** 26,629
- **Gene Count (Canonical FASTA):** 19,789
- **Taxonomy / Strain:** Bristol N2
- **NCBI Taxon ID:** 6239
- **Genome Assembly & Annotation:** GCA_000002985.3 (ENA/EMBL)

Arabidopsis thaliana (Mouse-ear cress)

- **Proteome ID:** UP000006548
- **Status:** Reference proteome
- **Total UniProtKB Entries:** 39,273
- **Gene Count (Canonical FASTA):** 27,496
- **Taxonomy / Strain:** cv. Columbia
- **NCBI Taxon ID:** 3702
- **Genome Assembly & Annotation:** GCA_000001735.2

Saccharomyces cerevisiae (Baker's yeast)

- **Proteome ID:** UP000002311
- **Status:** Reference proteome
- **Total UniProtKB Entries:** 6,067
- **Gene Count (Canonical FASTA):** 6,066
- **Taxonomy / Strain:** strain ATCC 204508 / S288c
- **NCBI Taxon ID:** 559292
- **Genome Assembly & Annotation:** GCA_000146045.2 (ENA/EMBL)

Escherichia coli (strain K12)

- **Proteome ID:** UP000000625
- **Status:** Reference proteome
- **Total UniProtKB Entries:** 4,403
- **Gene Count (Canonical FASTA):** 4,403
- **Taxonomy / Strain:** K12 / MG1655 / ATCC 47076
- **NCBI Taxon ID:** 83333
- **Genome Assembly & Annotation:** GCA_000005845.2 (ENA/EMBL)

%labelapp:commands

B Python Scripts Developed

This appendix provides a detailed overview of the Python scripts utilized throughout the Low-Complexity Region (LCR) benchmarking pipeline , organized by their respective figure execution modules.

B.1 Shared Preprocessing Scripts

These scripts handle the baseline cleaning and structural preparation of input genomic datasets shared across multiple downstream analysis steps .

00_prepare_bedfiles.py Prepares and standardizes input BED files from the raw directory into a format compatible with downstream processing .

01_cleanfasta_headers.py Cleans and standardizes structural headers within the raw input FASTA file to prevent formatting mismatches .

B.2 Figure 1: Baseline Properties (Length, Coverage, Composition)

Scripts responsible for parsing LCR extractions and profiling their core baseline characteristics .

run_pipeline_1.py Extracts raw regional sequences from the target FASTA database matching the structural coordinates provided in the BED files, while mapping global protein lengths .

gen_len_counts_2.py Calculates specific regional properties, calculating individual LCR sequence lengths and recording total LCR occurrences per protein .

gen_diversity_3.py Measures the information-theoretic complexity of extracted sequences by calculating their Shannon Entropy .

Gen_coverage_4.py Computes the fractional structural coverage of LCR sequences relative to total protein lengths across the dataset .

aa_composition_5.py Tabulates raw amino acid composition distributions across the extracted LCR fragments .

fig1A_length_hm.py Generates the visual length distribution heatmap (Figure 1A) .

fig1B_coverage_hm.py Generates the visual regional coverage heatmap (Figure 1B) .

fig1C_count_dist.py Generates the LCR count distribution plot (Figure 1C) .

fig1D_aa_comp.py Generates the amino acid composition profile plot (Figure 1D)

.

fig1E_entropybox.py Generates the Shannon Entropy boxplot matrix (Figure 1E) .

comb_fig1.py Compiles panels A through E into a single, unified multi-panel composite figure .

B.3 Figure 2: Consensus Regions & Peptide Motifs

Scripts focused on cross-tool consensus sequence extractions, mathematical overlays, and repetitive peptide motifs .

multiinter_01.py A native Python port of the `bedtools multiinter` command that calculates overlapping intervals across all tool outputs .

02_consensus_beds.py Converts multi-intersection coordinates into programmatic consensus structural tracking tracks .

getfasta.py Extracted targeted sequences from the cleaned FASTA using established consensus interval intersections .

04_substring_motif.py Scans consensus FASTA files to identify repeating substring structural motifs .

05_subfilter.py Evaluates and filters out highly redundant or weak sequence motifs based on statistical variance .

06_peptide_counts.py Tallies structural counts and occurrence rates for the discovered low-complexity peptide chains .

entropy.py Evaluates global Shannon Entropy metrics across consensus multi-tool intersections .

purity.py Derives sequence purity metrics based on localized single-amino-acid enrichment dynamics within intersections .

fig2a.py, fig2b.py, fig2c.py Plotting tools that output the respective peptide motif profiles, consensus entropy spreads, and structural sequence purity graphs .

B.4 Figure 3: Purity Metrics Across Tools

getfastaforfig3.py Slices tool-specific structural subsets across the clean FASTA master file to build comparative data blocks .

purity_calculation.py Computes micro-scale sequence purity coefficients independently for each evaluated tool .

fig3_plot.py Charts comparative tool-specific sequence purity distributions across a shared metric frame .

B.5 Figure 4: Inter-Tool Jaccard Overlaps

jaccard_similarity.py Computes an all-versus-all pair-wise Jaccard similarity index matrix across all active tool inputs .

fig4_plot.py Renders the final pair-wise similarity Jaccard matrix plot .

B.6 Figure 5: Sequence Mutation Profiling

mutation_metrics.py Extracts specific sequence mutation dynamic tracks and divergence markers across tool categories .

fig5_hm.py Dynamically generates individual mutation heatmaps tracking variance profiles across tool inputs .

Fig5_combined.py Combines individual tool mutation heatmaps into a single consolidated layout .

B.7 Figure 6: Windowed Classification & TPR/FPR Bounds

Scripts that build an analytical synthetic classification baseline utilizing slicing windows to evaluate tool recall performance metrics .

01_seq_parser.py Parses the master cleaned FASTA file into discrete, uniform sliding sequence tracking windows .

02_complexity_plot.py Evaluates localized sequence complexity steps across all generated sequence windows .

03_classification.py Formally classifies tracking windows based on analytical thresholds to create a reference category .

04_annotate.py Compiles windowed classification scores into functional genomic reference tracks .

protein_confusion.py Evaluates tool outputs against the compiled window references to generate standard multi-class confusion matrices .

reference_metrics.py Summarizes global validation indices across the constructed reference track datasets .

tpr_fpr.py Computes empirical True Positive Rates (TPR) and False Positive Rates (FPR) per evaluated software configuration .

plot_fig6.py Renders performance comparisons tracking gene lengths, LCR counts, windowed coverages, and entropy ratio variables .

fig6_combined.py Combines individual window performance components into a single multi-panel visual .

B.8 Figure 7 & Documentation Generation

buildsum.py Aggregates localized TPR/FPR performance markers to compile summary metrics .

plot_fig7.py Renders the ultimate summary bar plots evaluating comparative tool performance bounds .

generate_report.py Programmatically binds compiled figures together to automatically compile the final PDF benchmark document .

C Benchmark Results : Mouse

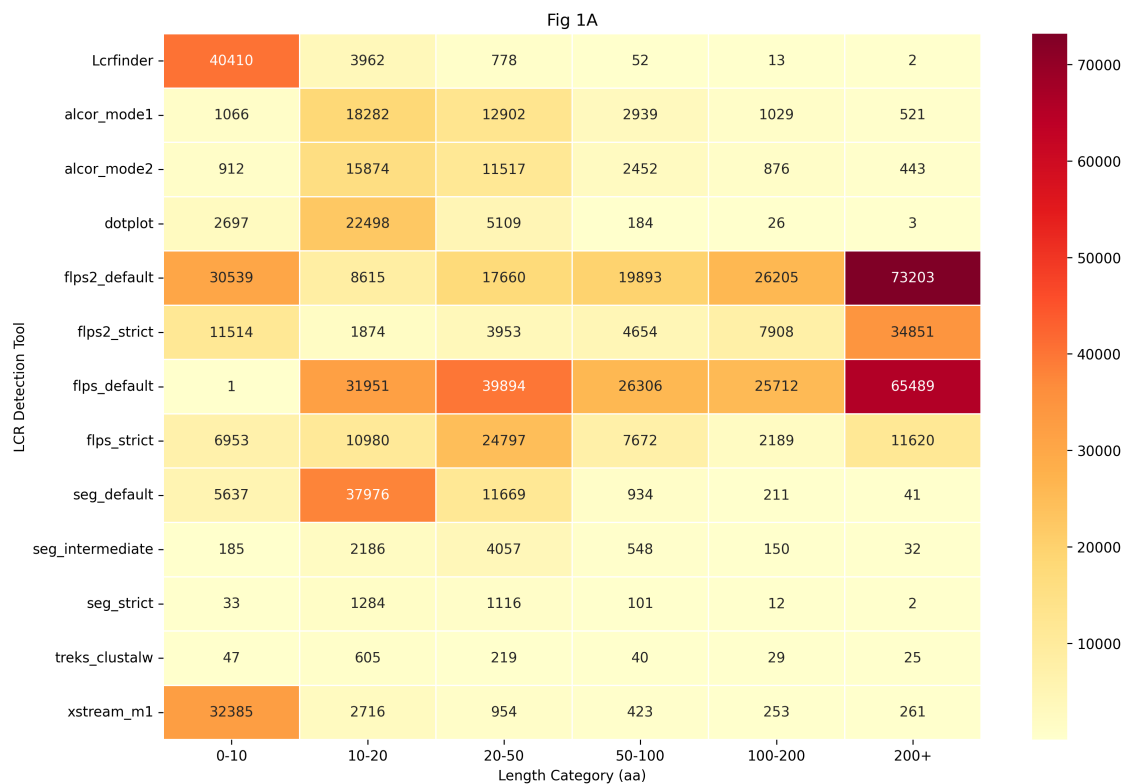


Figure C.1: LCR Count across length bins

Heatmap of counts across LCR length bins detected by each method across the proteome, with the y-axis listing the methods and the x-axis showing the LCR length bins (0-10, 10-20, 20-50, 50-100, 100-200 and above 200).

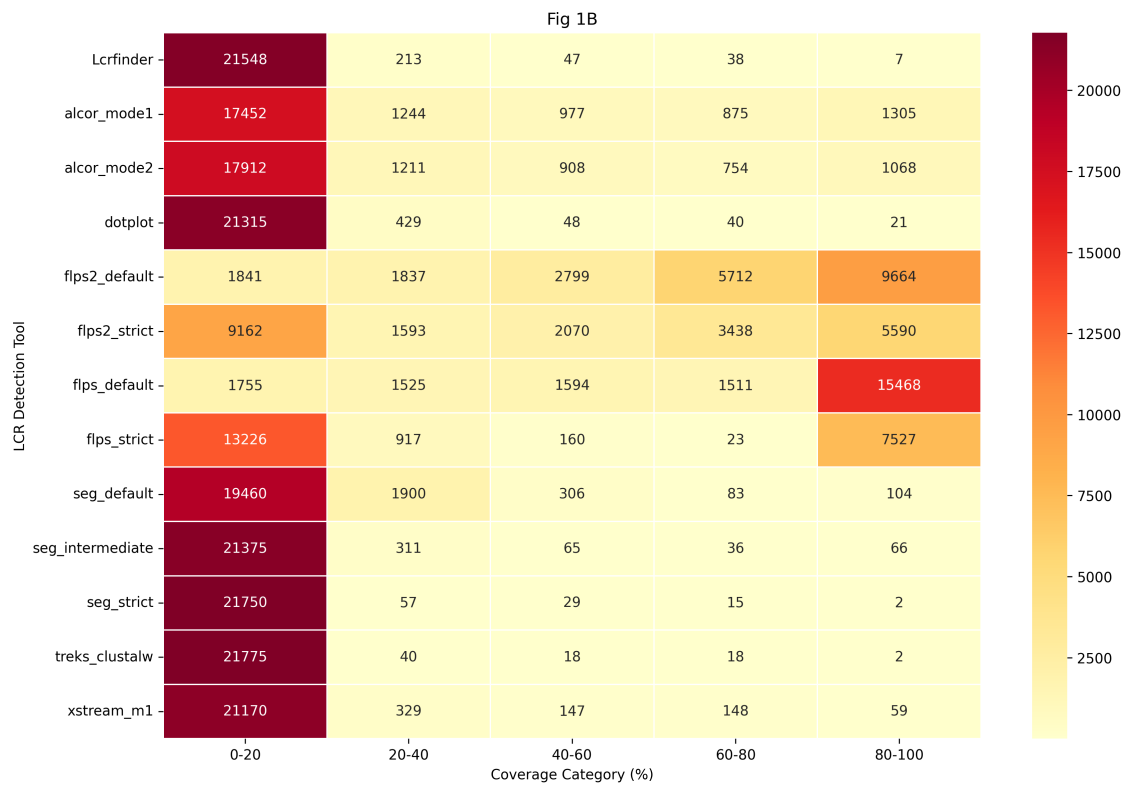


Figure C.2: Coverage Heatmap.

Heatmap of counts in percentage coverage bins (LCR coverage of the total gene length), grouped as 0-20, 20-40, 40-60, 60-80 and 80-100 per cent.

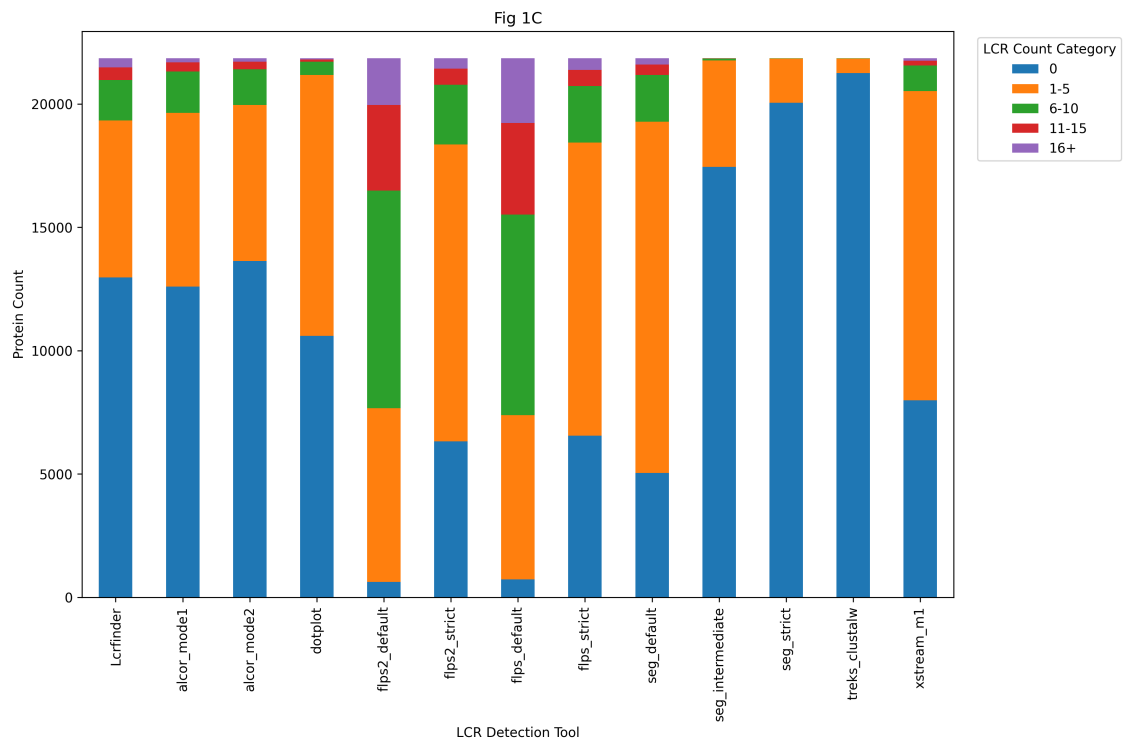


Figure C.3: Count Distribution.

Stacked bar plot comparing the number of LCRs detected per protein by each method, in categories of 0, 1-5, 6-10, 11-15 and above 15.

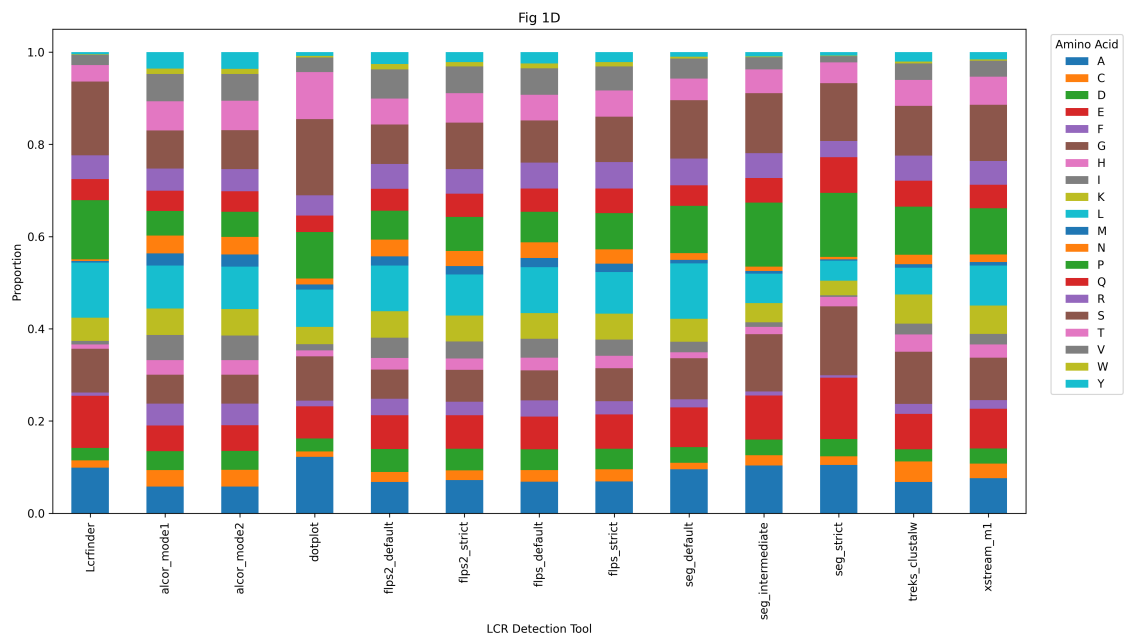


Figure C.4: Amino Acid Composition.

Stacked bar plot of the overall amino acid composition of LCRs detected by each method.

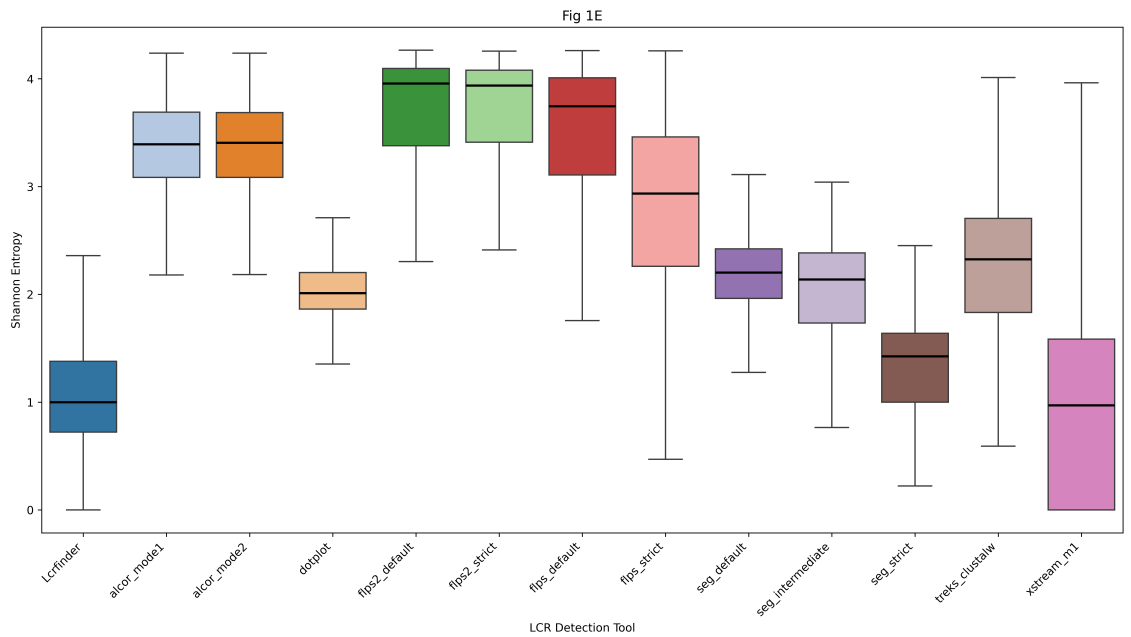


Figure C.5: Shanon Entropy Boxplot.

Boxplot comparing the overall Shannon entropy of LCRs detected by each method.

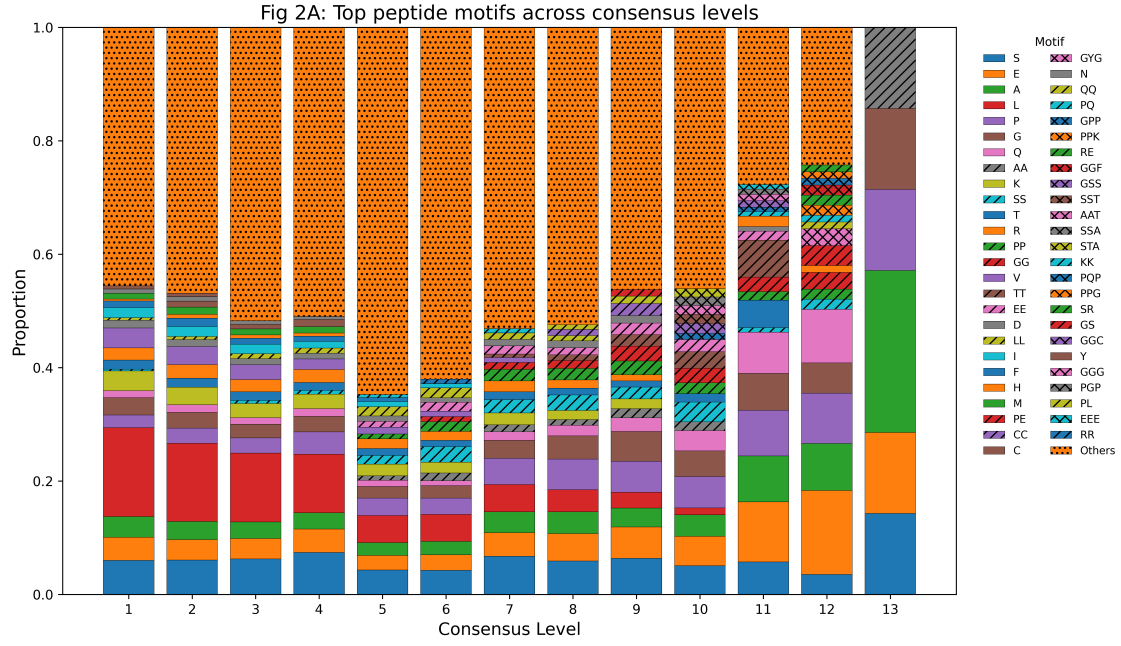


Figure C.6: Consensus regions and structural peptide motif variations

Stacked bar plot showing the proportional contribution of peptide motifs across LCRs grouped by the number of methods (n) detecting each region (consensus level, $n = 1-13$). Motifs include mono-, di- and tri-peptide compositions, with colours representing specific amino acids or repeating sequence patterns and hatching indicating peptide type. Regions detected by fewer methods are enriched in diverse and heterogeneous motif compositions, whereas higher-consensus regions show increased dominance of specific low-complexity motifs, reflecting stronger compositional bias.

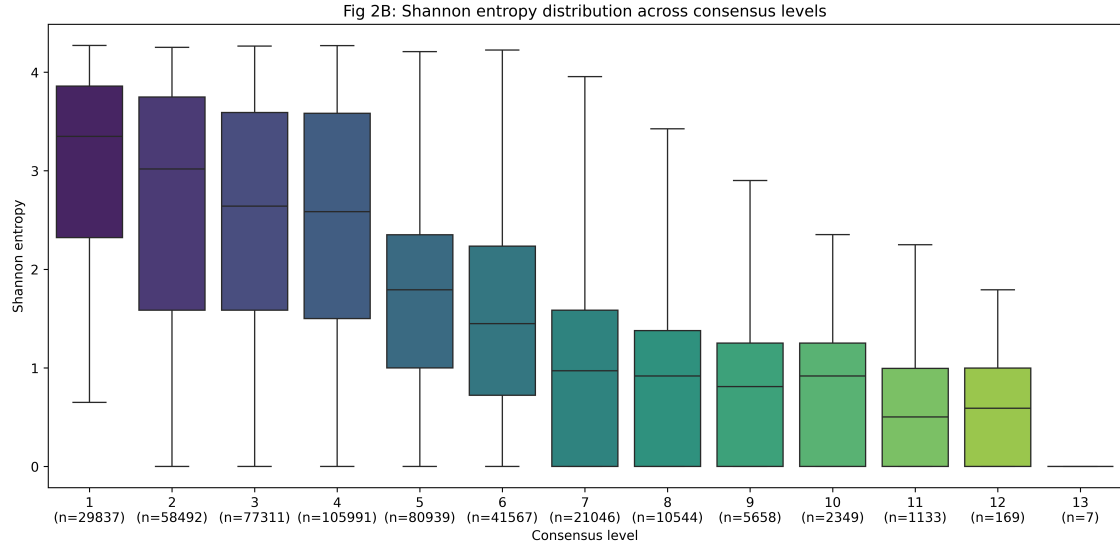


Figure C.7: Shanon Entropy Distribution across Consensus levels.

Boxplots of Shannon entropy for LCRs stratified by consensus level. Lower-consensus regions (detected by fewer methods) exhibit higher entropy, indicating greater sequence heterogeneity, while regions detected by multiple methods show progressively reduced entropy, consistent with increased repetitiveness and compositional constraint.

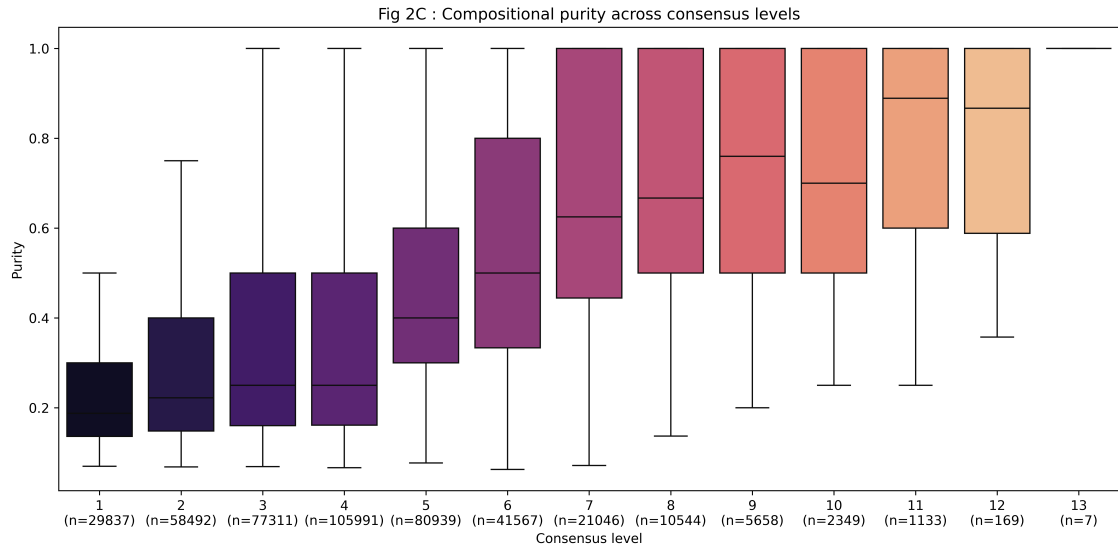


Figure C.8: Compositional Purity Across Consensus Levels.

Boxplots of compositional purity across the same consensus groups. Purity increases with the number of detecting methods, indicating that high-consensus LCRs are more compositionally homogeneous and dominated by fewer amino acid types.

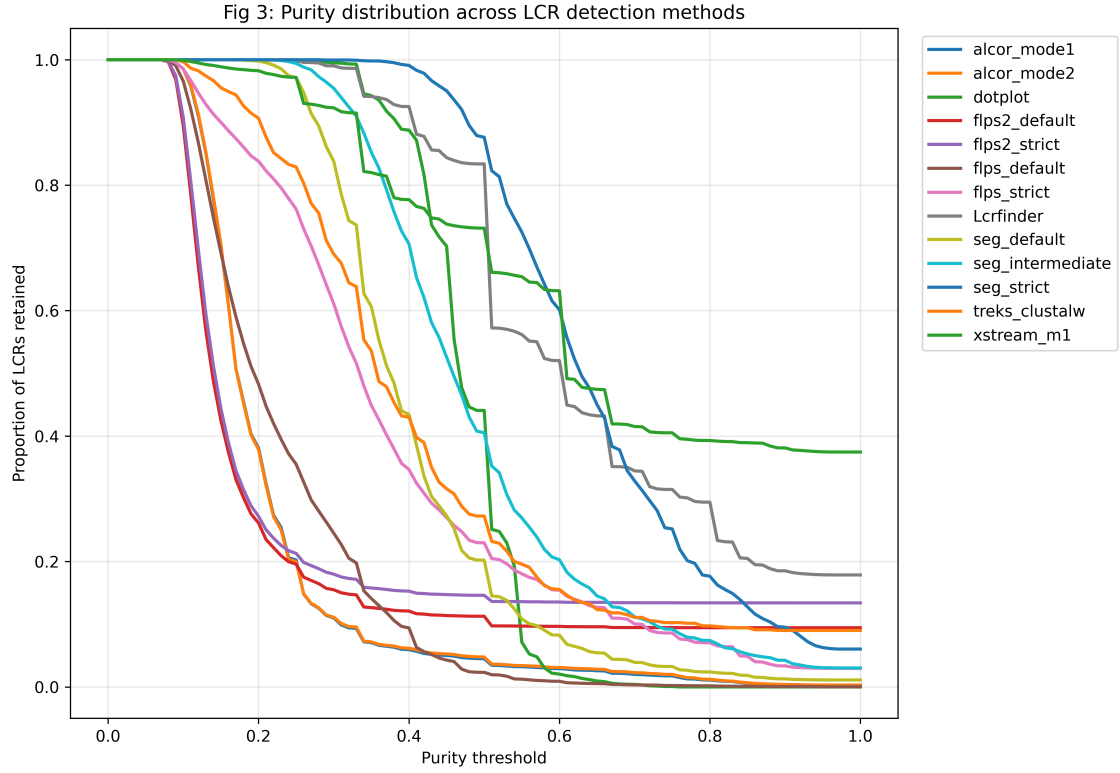


Figure C.9: Purity metrics distribution across individual tools.

The x-axis represents the purity level, defined as the percentage of the most dominant amino acid within each LCR, while the y-axis denotes the proportion of total LCRs retained at or above each purity threshold. Each line corresponds to a different method, as indicated by the colour-coded legend. Steeper declines indicate methods that detect fewer highly pure (compositionally biased) regions, whereas gradual slopes reflect identification of more homogeneous LCRs. This comparison emphasises how algorithms vary in their sensitivity to amino acid repetitiveness.

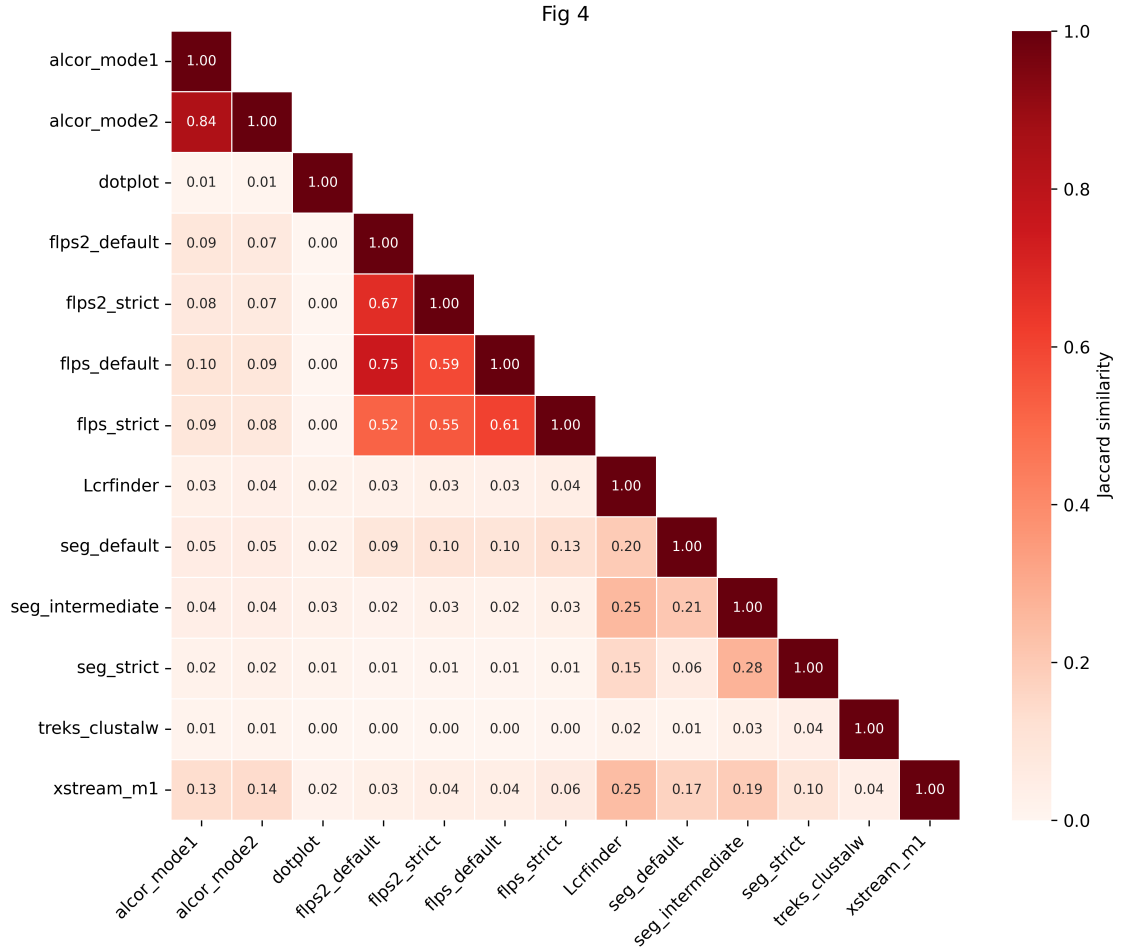


Figure C.10: Jaccard Similarity Matrix.

Jaccard similarity index for each pair of methods, calculated as the intersection divided by the union of detected LCR sets. The colour gradient (white to red) denotes increasing similarity, with diagonal values (1.0) representing self-comparisons. Together, this quantifies the extent of agreement and divergence among the evaluated detection algorithms.

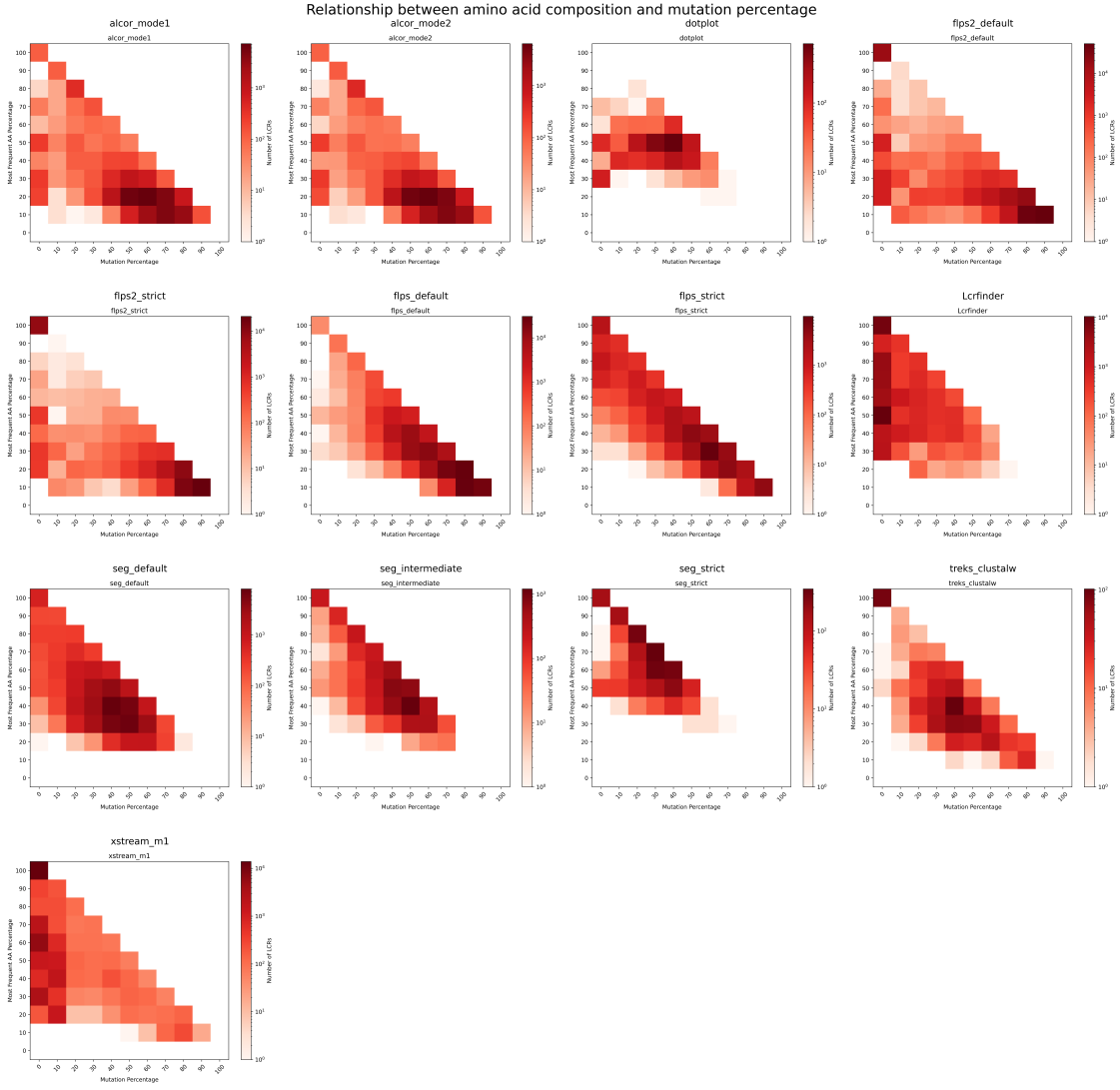


Figure C.11: Amino Acid Composition Vs. Mutation Percentage in LCRs.

Each panel corresponds to one detection method and illustrates the joint distribution of amino acid composition bias and sequence mutation percentage. The x-axis represents mutation percentage (fraction of variable residues within the LCR), and the y-axis shows the percentage of the most frequent amino acid. The colour gradient indicates the number of LCRs within each bin, as shown by the colour key. The diagonal trend visible across most methods indicates that higher amino acid dominance is typically associated with lower mutation percentages, reflecting stronger compositional homogeneity in repetitive regions.

This section assesses the performance of different tools in detecting an internally defined reference LCR set under varying conditions. True positive rate (TPR; left subpanels) and false positive rate (FPR; right subpanels) are plotted against four parameters (TPR/FPR reflect agreement with an operational internal reference, not biological truth). The x-axes represent increasing parameter values, while the y-axes show detection rates. Distinct colours denote individual tools, as indicated in the legend. Higher TPRs indicate more sensitive detection, whereas lower FPRs reflect higher specificity. These comparisons reveal how each method's performance varies with sequence length, LCR density, coverage, and compositional complexity.

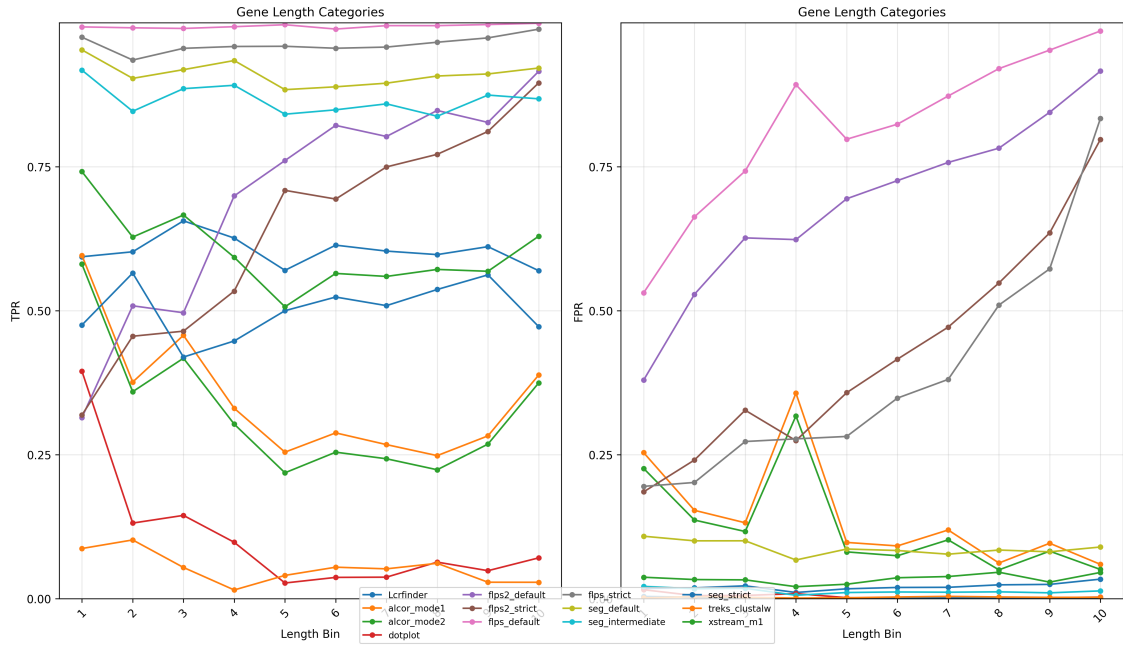


Figure C.12: Gene Length Categories

The parameter evaluated above represents the baseline classification rates mapped across distinct gene length categories.

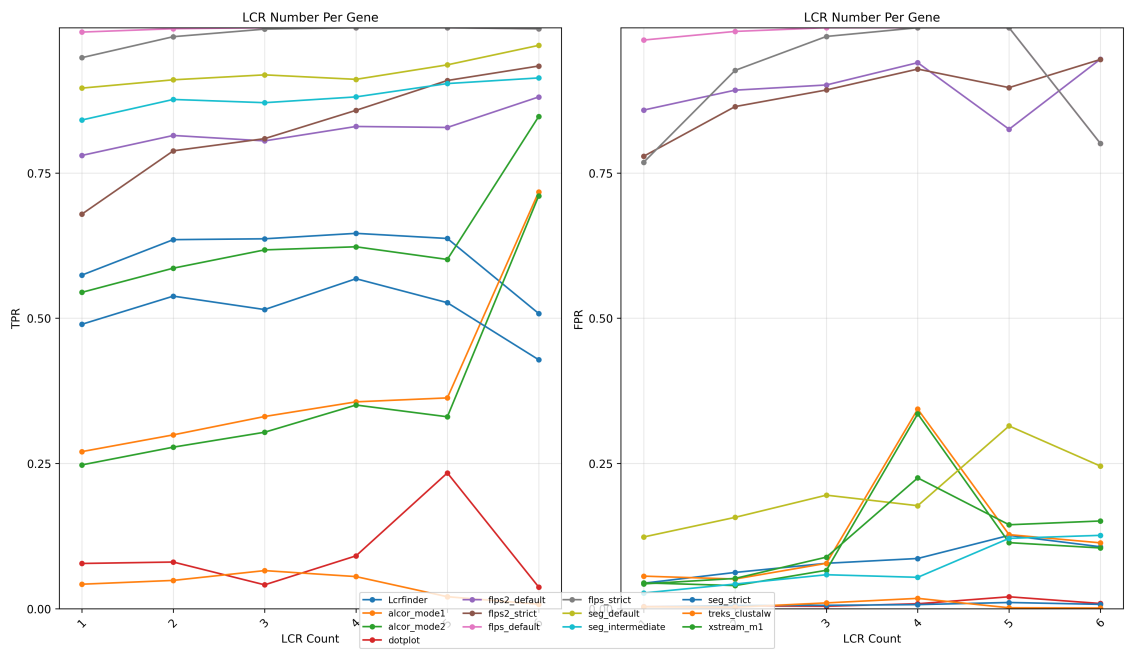


Figure C.13: No. of LCRs detected per gene.

The performance tracking layout above maps variation against the exact number of low-complexity domains isolated per single gene structure.

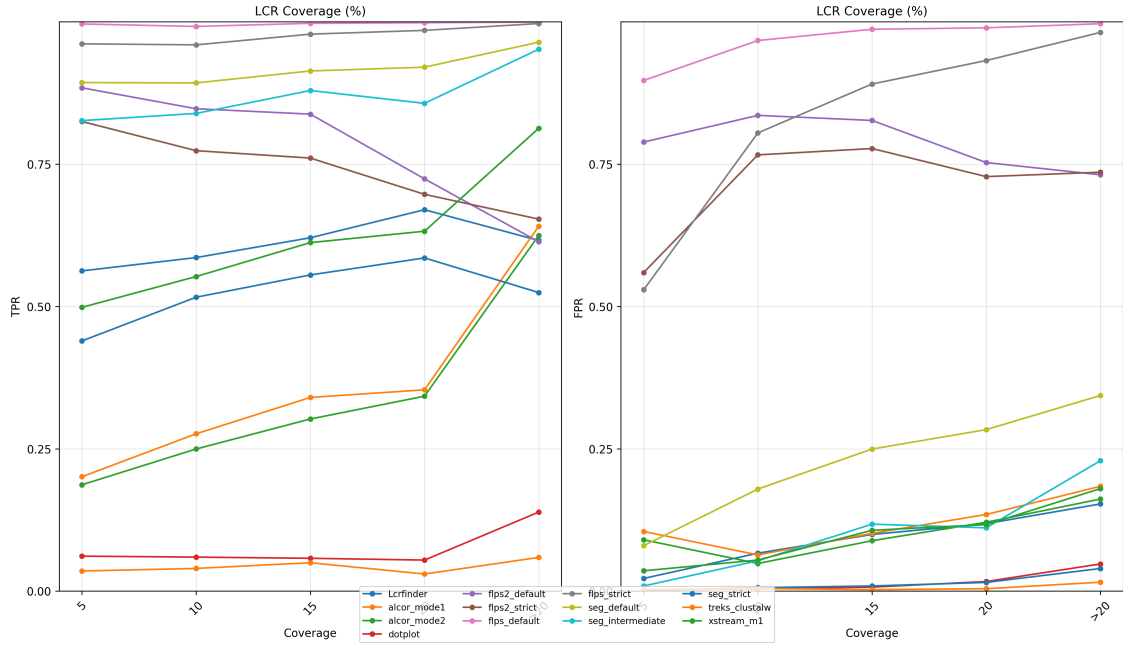


Figure C.14: LCR coverage percentage of the total gene length.

The rate configuration metrics profile tracking changes across sequence windows based on the overall percentage coverage scaling of the gene body.

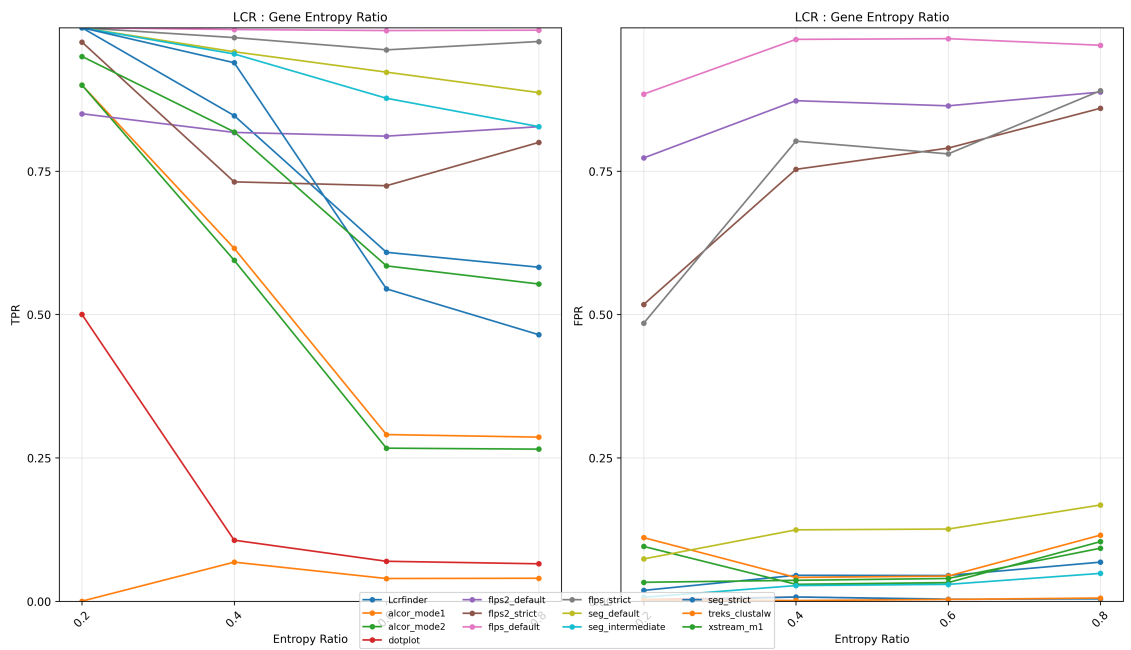


Figure C.15: LCR-to-gene entropy ratio

The comparative analysis tracks changes plotted alongside shifts in structural alignment against the calculated baseline sequence-to-gene complexity bounds.

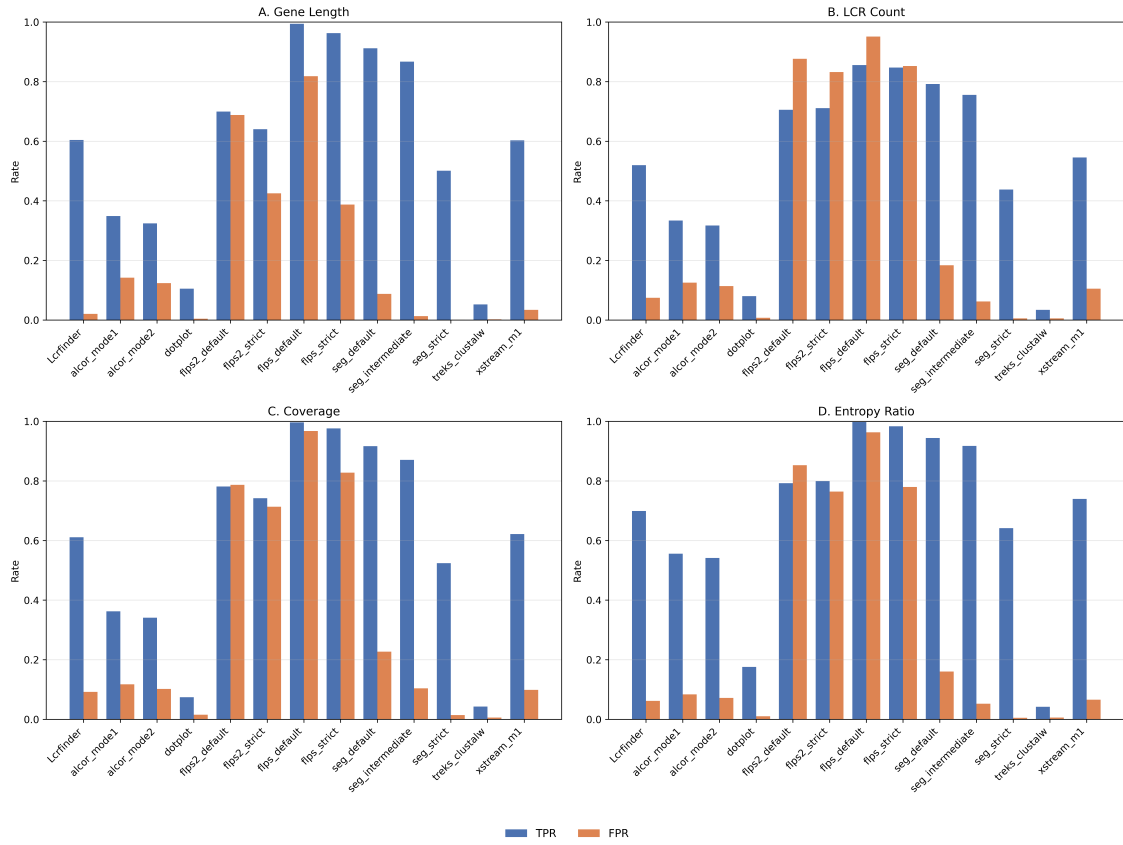


Figure C.16: Summary of benchmarking results across all evaluated LCR detection methods.

Final cross-comparison evaluating total metric distributions across all standard tools run through the execution sequence workflow parameters.